

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN MÔN HỌC
ĐỀ TÀI: TÌM HIỂU VÀ TRIỂN KHAI OSSEC

Môn học: NT534 – An toàn mạng máy tính nâng cao

Giảng viên hướng dẫn: Th.S. Đỗ Thị Phương Uyên

Thực hiện bởi nhóm 3, bao gồm:

- | | | |
|-------------------------|----------|-------------|
| 1. Dương Phước Nhật Nam | 23520968 | Trưởng nhóm |
| 2. Lê Minh | 23520928 | Thành viên |
| 3. Nguyễn Gia Luân | 23520896 | Thành viên |
| 4. Lương Hoàng Long | 23520879 | Thành viên |

MỤC LỤC

MỤC LỤC	2
DANH SÁCH HÌNH	4
DANH MỤC BẢNG	5
LỜI MỞ ĐẦU	6
Chương I. TỔNG QUAN	7
Chương II. ĐẶT VẤN ĐỀ	8
1. Ngữ cảnh.....	8
2. Đặt vấn đề.....	8
3. Sơ đồ hệ thống.....	9
Chương III. CƠ SỞ LÝ THUYẾT	10
1. OSSEC.....	10
2. Tailscale	10
3. ELK Stack	10
4. TheHive	11
5. VMware Workstation	12
Chương IV. TRIỂN KHAI – HIỆN THỰC HÓA ĐỀ TÀI.....	13
1. Kịch bản triển khai	13
2. Cấu hình chi tiết.....	15
2.1. OSSEC	15
2.2. Tailscale.....	16
2.3. ELK Stack	19
2.4. TheHive.....	27
CHƯƠNG V. KẾT QUẢ THỰC NGHIỆM.....	38
1. OSSEC.....	38
1.1. Active Response.....	38
1.2. File Integrity checking	40
1.3. Rootkit detection	41
2. Tailscale	42
3. ELK Stack	42

4. TheHive	43
CHƯƠNG VI. KẾT LUẬN	45
1. Kết quả đạt được.....	45
2. Hướng phát triển.....	45
NGUỒN THAM KHẢO	46

DANH SÁCH HÌNH

Hình 1. Sơ đồ hệ thống	9
Hình 2. TheHive Community Edition	12
Hình 3. Cài đặt gói Tailscale trên pfSense.....	17
Hình 4. Cấu hình định tuyến Tailscale trên pfSense.....	17
Hình 5. Cấu hình xác thực pfSense với Tailscale	17
Hình 6. Cấu hình rule cho kết nối thông qua Tailscale	18
Hình 7. Cấu hình quảng bá vùng mạng trên Tailscale.....	18
Hình 8. Màn hình xác nhận máy đăng ký vào vùng mạng quản lý của Tailscale	19
Hình 9. Hoạt động của ELK Stack	20
Hình 10. Giao diện của Kibana	23
Hình 11. Dữ liệu OSSEC gửi về trên Kibana	24
Hình 12. Giao diện tạo bảng	24
Hình 13. Cấu hình biểu đồ cột.....	25
Hình 14. Cấu hình biểu đồ tròn	26
Hình 15. Cấu hình biểu đồ cột ngang	27
Hình 16. Quy trình xử lý của TheHive	28
Hình 17. Khởi kiến trúc Docker chạy TheHive.....	28
Hình 18. Giao diện đăng nhập TheHive	29
Hình 19. API Key của tài khoản TheHive	29
Hình 20. Các cảnh báo được truyền tới TheHive	34
Hình 21. Tạo Case Template	34
Hình 22. Giao diện tạo và chỉnh sửa Case Template.....	35
Hình 23. Tạo Case từ Alert	35
Hình 24. Chọn Case Template cho Case muốn tạo	36
Hình 25. Giao diện tạo Case từ Case Template	37
Hình 26. Chương trình kiểm thử được thực hiện và bị chặn bởi OSSEC	39
Hình 27. Nhật ký sự kiện của OSSEC chủ động chặn tấn công bruteforce	39
Hình 28. Thông tin chặn IP ở iptables	40
Hình 29. Cấu hình trên máy chủ OSSEC	40
Hình 30. Đăng tải tệp mới lên máy chủ web	41
Hình 31. Nhật ký sự kiện của OSSEC cho kịch bản File Integrity checking.....	41
Hình 32. Chạy đoạn mã độc hại để kiểm thử tính năng phát hiện Rootkit	41
Hình 33. Nhật ký sự kiện phát hiện rootkit	41
Hình 34. Giao diện web hiển thị thông tin thiết bị đã cài đặt Tailscale.....	42
Hình 35. Giao diện trang chủ của Kibana sau khi cấu hình hoàn tất	43
Hình 36. Case mới đã được tạo	43
Hình 37. Danh sách các Task để xử lý trường hợp.....	44

DANH MỤC BẢNG

Bảng 1. Cài đặt OpenSSH Server cho kịch bản Active Response	15
Bảng 2. Cài đặt sshpass cho kịch bản Active Response	15
Bảng 3. Cài đặt DVWA cho máy chủ web	15
Bảng 4. Chỉnh sửa module Syscheck cho kịch bản File Integrity Monitoring	16
Bảng 5. Tắt module Syscheck cho kịch bản phát hiện rootkit	16
Bảng 6. Cấu hình module Rootcheck cho kịch bản phát hiện Rootkit.....	16
Bảng 7. Các lệnh tạo thư mục vào khoanh vùng nguy hiểm	16
Bảng 8. Cài đặt Tailscale trên máy cài ELK Stack.....	19
Bảng 9. Lệnh thêm thư viện mã nguồn của Elastic	20
Bảng 10. Lệnh cài đặt elasticsearch	20
Bảng 11. Cấu hình Elasticsearch	20
Bảng 12. Cấu hình tài nguyên mà Elasticsearch được phép sử dụng.....	21
Bảng 13. Cấu hình Logstash.....	22
Bảng 14. Cấu hình Kibana.....	22
Bảng 15. Cấu hình máy chủ OSSEC để gửi dữ liệu về Logstash	22
Bảng 16. Lệnh triển khai ELK Stack.....	23
Bảng 17. Script gửi cảnh báo từ máy chủ OSSEC sang TheHive.....	33
Bảng 18. Chương trình kiểm thử kịch bản Active Response	38

LỜI MỞ ĐẦU

Trong thời đại số hóa hiện nay, sự phát triển mạnh mẽ của công nghệ thông tin đã và đang mang lại nhiều lợi ích to lớn cho các tổ chức và doanh nghiệp. Tuy nhiên, đi liền với sự phát triển đó là sự gia tăng không ngừng của các mối đe dọa an ninh mạng. Các kỹ thuật tấn công mạng đang ngày càng trở nên tinh vi, phức tạp và có tính tổ chức cao hơn, đặc biệt là các chiến dịch nhắm trực tiếp vào thiết bị đầu cuối và hệ thống dịch vụ – vốn thường là mắt xích yếu nhất trong hệ thống bảo mật.

Các giải pháp bảo vệ truyền thống như tường lửa thường sẽ được triển khai trong hệ thống, nhưng thường sẽ không đủ để ngăn ngừa tấn công, đặc biệt khi kẻ tấn công đã xâm nhập được vào hệ thống. Điều này đặt ra yêu cầu có một hệ thống có thể giám sát các hệ thống dịch vụ và các thiết bị đầu cuối theo thời gian thực một cách tập trung, đồng thời có thể cung cấp giao diện trực quan giúp phân tích tình hình hiện tại của hệ thống, cũng như cho phép người phân tích có thể tiến hành phân tích và xử lý các sự cố xảy ra trong hệ thống.

Mục tiêu cốt lõi của đồ án là nghiên cứu cơ sở lý thuyết, nguyên tắc hoạt động của OSSEC, Tailscale, ELK Stack và TheHive, từ đó tiến hành triển khai cài đặt giải pháp này trên kiến trúc hệ thống đề xuất. Thông qua việc xây dựng và thực thi các kịch bản triển khai và thử nghiệm, nhóm mong muốn đánh giá khả năng của các công nghệ trên.

Mặc dù đã cố gắng hết sức trong quá trình nghiên cứu, nhưng do giới hạn về mặt thời gian cũng như kinh nghiệm thực tiễn, báo cáo chắc chắn không thể tránh khỏi những thiếu sót. Nhóm chúng em xin gửi lời cảm ơn chân thành đến Cô Đỗ Thị Phương Uyên đã tận tình giảng dạy, định hướng và cung cấp cho chúng em những nền tảng kiến thức quan trọng để hoàn thành đồ án. Nhóm rất mong nhận được những góp ý, chỉ bảo thêm từ Cô để đề tài nghiên cứu được hoàn thiện hơn.

Chương I. TỔNG QUAN

- Đề tài: Tìm hiểu và triển khai OSSEC.
- Nội dung: Nhóm đã tiến hành tìm hiểu và triển khai OSSEC trên hệ thống đề xuất, thiết kế và triển khai các kịch bản kiểm thử tính năng, cũng như mở rộng đề tài, biến hệ thống thành một hệ thống quản lý thông tin và sự kiện (SIEM – Security Information and Event Management) và quản lý trường hợp (Case Management) với Tailscale, ELK Stack và TheHive.
- Thời gian thực hiện: 3 tháng (02/2026 – 05/2026)

Chương II. ĐẶT VẤN ĐỀ

1. Ngữ cảnh

Trong bối cảnh hiện nay, khi hạ tầng mạng và hệ thống thông tin của tổ chức ngày càng được mở rộng, các rủi ro doanh nghiệp gặp phải trước nguy cơ bị tấn công mạng ngày càng tăng lên. Các hệ thống dịch vụ và thiết bị đầu cuối đang trở thành mục tiêu bị tấn công khai thác lỗ hổng, hay bị xâm nhập trái phép. Tuy các giải pháp truyền thống đã được triển khai như tường lửa để lọc lưu lượng không hợp lệ, hệ thống vẫn gặp phải rủi ro nếu như kẻ tấn công qua mặt được tường lửa và xâm nhập vào trong hệ thống. Vì vậy, cần thiết phải có một hệ thống giám sát trên thời gian thực các hành vi bất thường để kịp thời phát hiện và ngăn chặn, đồng thời phải có nền tảng quản lý sự kiện diễn ra và quản lý sự cố để từ đó có thể hiểu rõ tình hình an ninh của hệ thống hiện tại, cũng như phân tích được gốc rễ tấn công diễn ra như thế nào. Một giải pháp cho vấn đề trên là việc sử dụng OSSEC cho việc giám sát các thiết bị đầu cuối và dịch vụ, kết hợp ELK Stack và TheHive để quản lý thông tin, sự kiện và sự cố.

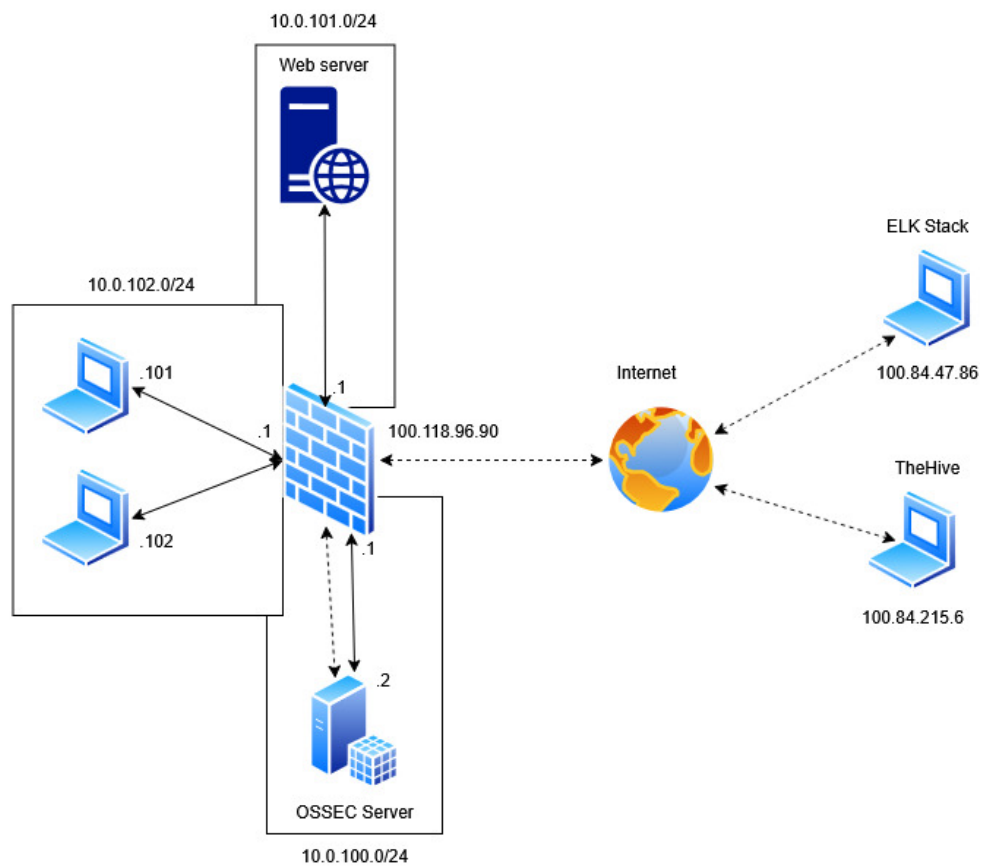
2. Đặt vấn đề

Đề tài được thực hiện dựa trên những vấn đề bao gồm:

- Làm thế nào để có thể theo dõi và phát hiện hành vi bất thường trên các thiết bị đầu cuối theo thời gian thực một cách tập trung?
- Làm thế nào để có thể tổng hợp và trực quan hóa tình hình hiện tại của hệ thống?
- Làm thế nào để có thể kết nối tới các dịch vụ bên ngoài mạng nội bộ an toàn?
- Cần phải biết xử lý sự cố, cũng như cung cấp môi trường điều tra về sự cố như thế nào để từ đó có thể cải thiện khả năng bảo mật của hệ thống?

Để giải quyết những vấn đề trên, đồ án sẽ tập trung tìm hiểu và triển khai giải pháp OSSEC, kiểm thử tính năng, cũng như kết hợp thêm với Tailscale, ELK Stack và TheHive để bảo vệ thiết bị đầu cuối, kết nối dịch vụ từ xa an toàn, tập trung hóa nhật ký sự kiện, và cung cấp môi trường cho phép quản lý và xử lý sự cố.

3. Sơ đồ hệ thống



Hình 1. Sơ đồ hệ thống

Để thực hiện đề tài, nhóm sẽ đề xuất sơ đồ hệ thống như ở hình trên, cụ thể:

- Mạng bên ngoài và mạng nội bộ được chia cách bởi tường lửa pfSense.
- Mạng nội bộ bao gồm hệ thống được bảo vệ bởi OSSEC, bao gồm 3 mạng con: Mạng 10.0.100.0/24 chứa máy chủ OSSEC, mạng 10.0.101.0/24 chứa máy chủ web, mạng 10.0.102.0/24 chứa các thiết bị đầu cuối.
- Mạng bên ngoài bao gồm hai máy được cài đặt ELK Stack và TheHive, hai máy này sẽ được kết nối tới máy chủ OSSEC để nhận nhật ký sự kiện thông qua Tailscale.

Chương III. CƠ SỞ LÝ THUYẾT

1. OSSEC

OSSEC là một nền tảng chuyên để giám sát hệ thống, được tích hợp các chức năng của hệ thống phát hiện xâm nhập cho thiết bị đầu cuối (HIDS - host-based intrusion detection), giám sát nhật ký và SIEM. Cụ thể, OSSEC cung cấp các tính năng sau:

- File Integrity checking: Kẻ tấn công thường sẽ thay đổi các tệp hệ thống sau khi xâm nhập vào, nên mục tiêu của tính năng này sẽ cố gắng phát hiện thay đổi của tệp trong thư mục được theo dõi và sẽ cảnh báo tới quản trị viên.
- Log Monitoring: Tính năng chính mà OSSEC sử dụng để có thể cảnh báo đến người dùng bằng cách giám sát, phân tích và tương quan các nhật ký sự kiện của thiết bị đầu cuối.
- Rootkit detection: Kẻ tấn công thường sẽ ẩn đi tiến trình mã độc chính, hoặc ngụy trang bằng một tiến trình khác, nhưng OSSEC vẫn sẽ phát hiện được thay đổi trên hệ thống, và nếu thay đổi đó được nghi vấn là rootkit, OSSEC sẽ thông báo đến quản trị viên.
- Active response: Khi có cảnh báo ở mức độ nào đó được phát ra, OSSEC sẽ ngay lập tức có thể chặn hành vi độc hại.

2. Tailscale

Tailscale là nền tảng kết nối Zero Trust dựa trên danh tính máy kết nối, cho phép kết nối trên nhiều môi trường khác nhau như làm việc từ xa, đám mây, CI/CD, AI,... Tailscale giải quyết nhu cầu của một môi trường làm việc phân tán, phù hợp với nhu cầu đồ án, phải sử dụng nhiều máy thật để có thể triển khai hệ thống, đồng thời cũng hỗ trợ tốt khi nhu cầu hệ thống được mở rộng mà không cần cấu hình phức tạp.

Được dựa trên giao thức WireGuard, Tailscale tạo kết nối VPN peer-to-peer tới các máy được kết nối, giúp cải thiện độ trễ kết nối so với việc sử dụng một máy chủ tập trung, quản lý lưu lượng kết nối của các máy khác được kết nối.

3. ELK Stack

ELK được viết tắt từ 3 chữ cái đầu của Elasticsearch, Logstash và Kibana, đây là một tập hợp công cụ các mã nguồn mở kết hợp để tạo thành một nền tảng quản lý, phân tích và tìm kiếm dữ liệu tập trung (đặc biệt là tệp lưu nhật ký sự kiện) cực kỳ mạnh mẽ và phổ biến hiện nay.

Elasticsearch: Đóng vai trò trong việc tìm kiếm và phân tích dạng RESTful, phân tán dựa trên nền tảng Apache Lucene-là nơi lưu trữ toàn bộ dữ liệu do Logstash gửi đến. Elasticsearch có cơ chế lưu trữ dữ liệu dưới dạng các tài liệu JSON, điều này giúp việc tìm kiếm trở nên nhanh hơn (gần như là ngay lập tức) thông qua việc sử dụng

Inverted Index. Điểm mạnh của Elasticsearch là khả năng mở rộng ngang nhờ cơ chế chia nhỏ dữ liệu và nhân bản, ngoài ra còn hỗ trợ những truy vấn phức tạp và phân tích thống kê số lượng lớn dữ liệu trong thời gian ngắn.

Logstash: Đóng vai trò là bộ thu thập và xử lý dữ liệu, thực hiện tiếp nhận dữ liệu từ nhiều nguồn khác nhau, thực hiện biến đổi và chuẩn hóa dữ liệu trước khi đưa tới nơi lưu trữ. Logstash sẽ thực hiện tiếp nhận các log từ nhiều nguồn (file, syslog, database, http,...) sau đó tiến hành phân tích cú pháp, lọc, làm giàu dữ liệu (phân tích IP ra vị trí địa lý, chuyển chuỗi thành số,...) cuối cùng là gửi dữ liệu đã xử lý đến đích (thông thường là Elasticsearch kể trên). Logstash được biết tới với khả năng chuyển đổi cấu trúc dữ liệu cực mạnh mẽ.

Kibana: Đóng vai trò làm giao diện trực quan hóa dữ liệu, là tầng giao diện chạy trên nền web, giúp người dùng tương tác trực tiếp với dữ liệu được lưu trữ trong Elasticsearch. Kibana sẽ thực hiện kết nối với Elasticsearch để truy vấn dữ liệu và hiển thị chúng dưới dạng các biểu đồ sinh động. Kibana mạnh ở chỗ có thể tạo ra được các giao diện (tạo Dashboard tùy ý như biểu đồ cột, đường, tròn,...). Hỗ trợ giám sát hệ thống theo thời gian thực, quản lý cấu hình cluster của Elasticsearch và phân quyền người dùng một cách trực quan.

Giờ đây việc ngoài ELK cũ ta có thêm sự xuất hiện của **Beats**, giúp cắt giảm thời gian, tài nguyên cài đặt Logstash trên từng máy chủ để thu thập log (thu thập dữ liệu thô). Beats là các Agent siêu nhẹ cài trực tiếp trên các máy chủ nguồn để thu thập dữ liệu và đẩy về Logstash hoặc trực tiếp vào Elasticsearch. Ta có các loại Beats phổ biến bao gồm:

- **Filebeat:** Thu thập các file log (System log, Application log)
- **Metricbeat:** Thu thập thông số hệ thống (CPU, RAM, Disk).
- **Packetbeat:** Giám sát dữ liệu mạng
- **Winlogbeat:** Thu thập event log trên Windows.

4. TheHive

TheHive là một Nền tảng Phản ứng Sự cố Bảo mật (Security Incident Response Platform – SIRP) thương mại. Vai trò cốt lõi của TheHive là hệ thống Quản lý Trường hợp (Case Management) tập trung. Nền tảng này giúp gom cụm, tổ chức và điều phối toàn bộ vòng đời của một sự cố an ninh mạng, từ lúc tiếp nhận cảnh báo ban đầu cho tới khi đóng trường hợp.



Hình 2. TheHive Community Edition

Các chức năng cốt lõi của TheHive bao gồm:

- Quản lý cảnh báo: Tiếp nhận cảnh báo từ nhiều nguồn đa dạng (SIEM, EDR, email, ...). Các cảnh báo có thể được lọc, gom nhóm và đánh giá mức độ ưu tiên.
- Quản lý trường hợp: Cho phép xác định một cảnh báo thành một trường hợp tấn công (case). Tại đây, người quản trị có thể phân công nhiệm vụ cho các chuyên gia phân tích khác nhau.

TheHive có khả năng xử lý thời gian thực. Nhiều nhà phân tích có thể cùng thao tác trên một trường hợp. Quản trị viên có thể cấu hình sẵn các khuôn mẫu cho từng kịch bản tấn công cụ thể (như Ransomware, DoS, ...). Khi phát hiện đúng kịch bản tấn công đó, TheHive sẽ tự động tạo một trường hợp tấn công với các quy trình phản ứng theo chuẩn đã được định nghĩa sẵn. Ngoài ra, TheHive còn được xây dựng với tiêu chí API là trên hết. Mọi thao tác người dùng thực hiện trên giao diện web (GUI) đều có thể được gọi thông qua REST API, giúp hệ thống cực kỳ thân thiện với việc lập trình và tự động hóa.

TheHive không phải Giải pháp Điều phối và Tự động hóa Phản ứng An ninh (SOAR) như n8n hay Shuffle. TheHive là một nền tảng Quản lý Trường hợp (Case Management), một phần khác biệt hoàn toàn với hệ thống SOAR. Tuy TheHive có khả năng tự động phản ứng qua Cortex Responders, nó chỉ có thể thực hiện được các hành động riêng lẻ, không thể thiết kế luồng hoạt động phản ứng như các giải pháp SOAR thông thường. Vậy nên, không thể xem TheHive là một giải pháp thay thế cho SOAR được.

5. VMware Workstation

VMware Workstation là nền tảng cung cấp công nghệ tạo ra các máy ảo có hệ điều hành khác nhau trên một máy thật. VMware Workstation cũng cho phép tạo ra các adapter mạng để các máy ảo có thể giao tiếp trong cùng một mạng cô lập. Đây chính là những điều kiện cần thiết để có thể triển khai hệ thống đề xuất.

Chương IV. TRIỂN KHAI – HIỆN THỰC HÓA ĐỀ TÀI

1. Kịch bản triển khai

Tương ứng với các tính năng của OSSEC, nhóm sẽ tiến hành xây dựng kịch bản kiểm thử cho từng tính năng, ngoại trừ tính năng Log Monitoring, bởi vì tính năng này đã hiện diện trong suốt quá trình tìm hiểu và triển khai các kịch bản kiểm thử. Cụ thể các kịch bản như sau:

- Active Response:
 - Đầu tiên là về mục tiêu của kịch bản này, nhóm thực hiện kịch bản này nhằm kiểm chứng khả năng OSSEC tự động nhận diện mỗi đe dọa và thực thi các biện pháp ngăn chặn theo thời gian thực mà không cần sự can thiệp thủ công của quản trị viên
 - Nhóm đóng vai kẻ tấn công, sử dụng một danh sách mật khẩu để tiến hành chiến dịch tấn công dò thông tin đăng nhập vào dịch vụ SSH của máy chủ mục tiêu. Tính năng Log Monitoring của OSSEC sẽ bắt được các bản ghi lỗi xác thực liên tiếp. Ngay khi số lần đăng nhập sai vượt mức ngưỡng cho phép (tương ứng với Rule 5720), cơ chế Active Response sẽ ngay lập tức được kích hoạt. Hệ thống sẽ tự động thực thi lệnh cấm cửa địa chỉ IP của kẻ tấn công trên tường lửa của máy mục tiêu, cắt đứt hoàn toàn luồng tấn công.
- File Integrity checking:
 - Nhóm thực hiện kịch bản này nhằm đánh giá khả năng của OSSEC (thông qua module Syscheck) trong việc giám sát, phát hiện và cảnh báo sớm các hành vi trái phép như tạo mới hoặc sửa đổi nội dung các tệp tin trong những thư mục nhạy cảm.
 - Nhóm tiến hành cấu hình OSSEC Agent theo dõi thời gian thực một thư mục dịch vụ web (ví dụ: thư mục tải lên trong phần File Upload của ứng dụng DVWA). Tiếp đó, mô phỏng hành vi của hacker lên lút tải lên một tệp tin mới và tiến hành thay đổi nội dung bên trong tệp tin này nhằm chèn mã độc. Thông qua việc liên tục đối chiếu mã băm (MD5, SHA1, SHA256) của các tệp tin so với cơ sở dữ liệu gốc, hệ thống OSSEC Server sẽ ghi nhận và phát ra các cảnh báo ở mức độ cao. Cụ thể, phát hiện tệp tin mới bị thêm vào hệ thống (Rule 554) và cảnh báo tính toàn vẹn của tệp bị thay đổi (Rule 550), đồng thời chỉ ra chính xác vị trí dòng văn bản bị tác động.
- Rootkit detection:
 - Để hiểu rõ được khả năng của OSSEC trước tiên ta cần tìm hiểu rootkit. Rootkit là một loại phần mềm độc hại được thiết kế với mục đích chiếm quyền truy cập cấp cao nhất (root hoặc administrator) vào hệ thống và chủ

động che giấu sự hiện diện của chính nó cũng như các tiến trình độc hại khác, điểm khác biệt của nó ở khả năng ẩn mình tinh vi, nó sẽ không xuất hiện trong danh sách tiến trình, không để lại tiến trình mạng rõ ràng và thường qua mặt được các phần mềm Antivirus truyền thống. Rootkit can thiệp sâu vào tầng nhân, sửa đổi các cuộc gọi hệ thống để trả về thông tin giả cho người quản trị.

- OSSEC có công cụ tích hợp sẵn có tên là Rootcheck, chuyên để kiểm tra định kỳ (mặc định là 12 tiếng). OSSEC sẽ thực hiện **đối chiếu bất thường hệ thống** (chạy các cuộc gọi hệ thống trực tiếp so sánh kết quả trả về với kết quả của các lệnh tiêu chuẩn những lệnh bị rootkit sửa đổi sẽ bị phát hiện ở bước này). Không dừng lại ở đó OSSEC thực hiện **quét chữ ký dựa trên cơ sở dữ liệu** bởi vì sở hữu một cơ sở dữ liệu nội bộ chứa chữ ký của các loại Rootkit chính vì thế nó thực hiện kiểm tra file độc hại bằng cách quét các đường dẫn hệ thống để tìm kiếm các file, thư mục hoặc liên kết thường được tạo ra bởi các bộ Rootkit nổi tiếng, ngoài ra còn thực hiện **quét các file ẩn và thư mục bất thường** nhằm bắt được hành vi ẩn náu trong các phân vùng hệ thống mà người dùng ít khi kiểm tra kỹ, OSSEC thực hiện quét /dev, file hệ thống để tìm kiếm những dấu hiệu file bất thường. Cuối cùng là **tích hợp chặt chẽ với Syscheck kiểm tra song song với Rootcheck** giúp hạn chế nhưng khả năng cố gắng ghi đè hoạt sửa file hệ thống các khóa Registry để duy trì quyền thiết lập khi khởi động lại, Syscheck sẽ lập tức phát hiện ra sự thay đổi về mã băm (MD5/SHA256) và gửi cảnh báo về Server trung tâm.
- Kịch bản sẽ là ta thực hiện chạy đoạn lệnh thực hiện ghi file vào những phân vùng ẩn bị Rootcheck kiểm soát gắt gao nhằm thực hiện demo để thấy được khả năng phát hiện nghi ngờ rootcheck của OSSEC. Sau khi tạo các file bất thường ta tiến hành kiểm tra rootkit thông qua cơ chế rootcheck.

Ngoài ra, để có thể mở rộng hệ thống thành một nền tảng SIEM và Case Management, nhóm tiến hành tích hợp thêm công nghệ ELK Stack để đóng vai trò SIEM, và TheHive để có thể cải tiến hơn từ tính năng Active Response của OSSEC, cụ thể:

- Tailscale: Đồ án sẽ sử dụng công cụ này để cài đặt lên tường lửa pfSense (tường lửa sẽ đóng vai trò định tuyến vào mạng nội bộ) và hai máy được cài đặt ELK Stack và TheHive, thiết lập kết nối mạng bảo mật cho hệ thống được triển khai.
- ELK Stack: Đồ án sẽ sử dụng công cụ này để thu thập cảnh báo từ máy chủ OSSEC, sau đó sẽ trực quan hóa và hiển thị tình hình bảo mật của hệ thống với những đồ thị trực quan trên trang chủ của Kibana.

- TheHive: Đồ án sẽ sử dụng công cụ này để tạo và quản lý các Trường hợp cảnh báo (Case). Ngoài ra, TheHive còn được sử dụng để tạo và định nghĩa các Mẫu Trường hợp (Case Template), từ đó đề xuất các bước xử lý ứng với mỗi Trường hợp tấn công.

2. Cấu hình chi tiết

2.1. OSSEC

Lưu ý: Khi thực hiện sửa đổi các module của OSSEC, các thay đổi sẽ được thực hiện trên tệp `/var/ossec/etc/ossec.conf`.

2.1.1. Active Response

Chuẩn bị môi trường trên máy mục tiêu. Cài đặt và khởi động dịch vụ OpenSSH Server:

```
# Cài đặt dịch vụ SSH
sudo apt-get update
sudo apt-get install openssh-server -y

# Khởi động và cho phép SSH chạy ngầm cùng hệ thống
sudo systemctl start ssh
sudo systemctl enable ssh
```

Bảng 1. Cài đặt OpenSSH Server cho kịch bản Active Response

Tiếp theo ta cần cài đặt công cụ sshpass để truyền mật khẩu tự động:

```
sudo apt update
sudo apt install sshpass -y
```

Bảng 2. Cài đặt sshpass cho kịch bản Active Response

2.1.2. File Integrity Monitoring

Chuẩn bị môi trường Web mục tiêu (DVWA):

```
sudo apt-get update
sudo apt-get install apache2 mariadb-server php php-mysql php-gd -y

cd /var/www/html/
sudo git clone https://github.com/digininja/DVWA.git

sudo chown -R www-data:www-data /var/www/html/DVWA/
sudo chmod -R 777 /var/www/html/DVWA/hackable/uploads/

sudo systemctl start apache2
```

Bảng 3. Cài đặt DVWA cho máy chủ web

Cấu hình module Syscheck trên OSSEC Agent:

```
<syscheck>
  <frequency>60</frequency>
  <alert_new_files>yes</alert_new_files>
  <directories check_all="yes" report_changes="yes"
realtime="yes">/var/www/html/DVWA/hackable/uploads</directories>
</syscheck>
```

Bảng 4. Chính sửa module Syscheck cho kịch bản File Integrity Monitoring

2.1.3. Rootkit Detection

Ở phần này nhằm tối ưu thời gian phát hiện rootcheck thì ta sẽ thực hiện :

```
<syscheck>
  <disabled>yes</disabled>
....
....
</syscheck>
```

Bảng 5. Tắt module Syscheck cho kịch bản phát hiện rootkit

Bởi vì theo như tìm hiểu Syscheck sẽ thực hiện quét song song với rootcheck và quét bao quát khá lâu chính vì thế ta thực hiện tắt tính năng syscheck nhằm giảm thời gian quét được rootcheck phục vụ cho việc kiểm thử tính năng.

```
<rootcheck>
  <disabled>no</disabled>
  <frequency>120</frequency>
....
....
....
</rootcheck>
```

Bảng 6. Cấu hình module Rootcheck cho kịch bản phát hiện Rootkit

Ở đây ta cấu hình cho Rootcheck scan với chu kỳ 120s sau khi khởi chạy thay vì chờ đợi chu kỳ 12 tiếng mặc định như thông thường.

Sau khi cấu hình xong ta chạy đoạn lệnh độc hại bên dưới nhằm tạo ra những thư mục vào phần khoanh vùng nguy hiểm mà OSSEC đang để ý tới như đã nói ở phần trước, điều này chỉ mang tính chất thử nghiệm tính năng Rootcheck của OSSEC.

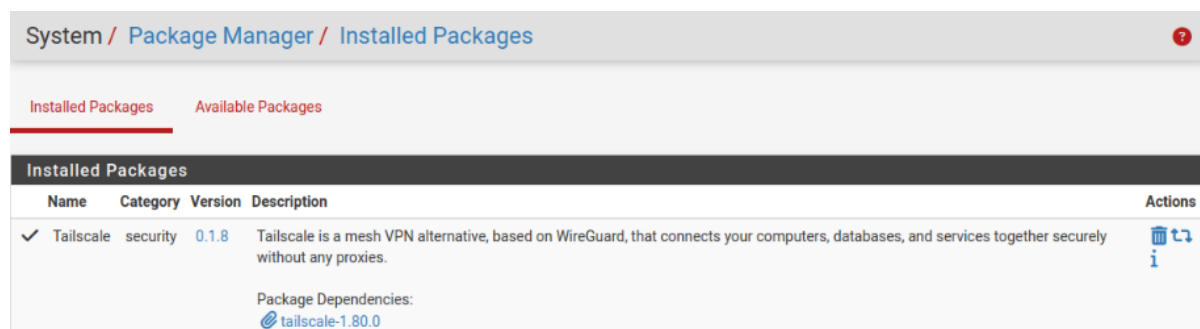
```
sudo mkdir -p /usr/lib/libsh
sudo touch /usr/lib/libsh/hidden
sudo touch /dev/ptyxx
```

Bảng 7. Các lệnh tạo thư mục vào khoanh vùng nguy hiểm

2.2. Tailscale

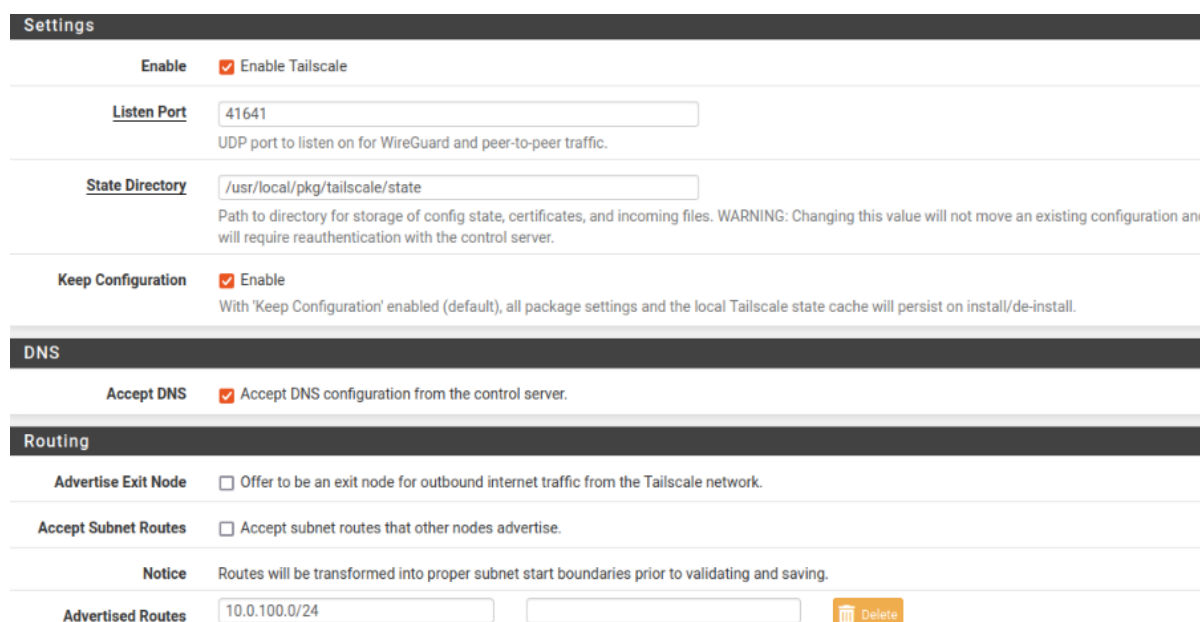
Cài đặt ở tường lửa pfSense:

- Vào System -> Package Manager và cài đặt gói Tailscale:



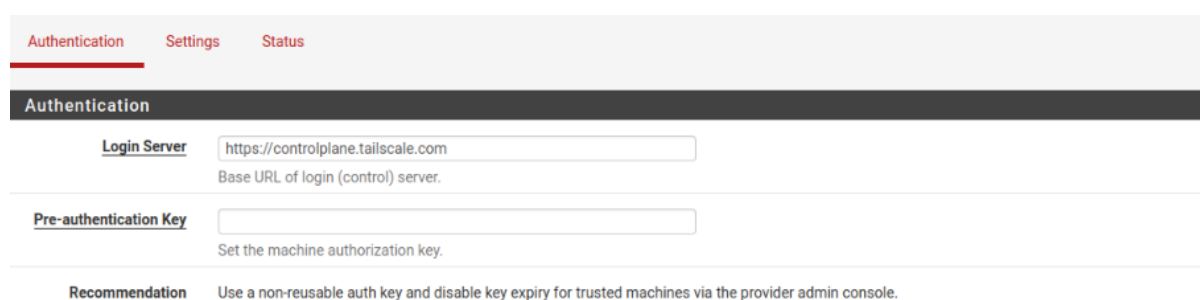
Hình 3. Cài đặt gói Tailscale trên pfSense

- Vào VPN -> Tailscale -> Settings, tiến hành kích hoạt Tailscale, đồng thời thêm dải mạng 10.0.100.0/24 để pfSense có thể định tuyến máy cài ELK Stack và TheHive vào máy chủ OSSEC ở mục Routing -> Advertised Routes:



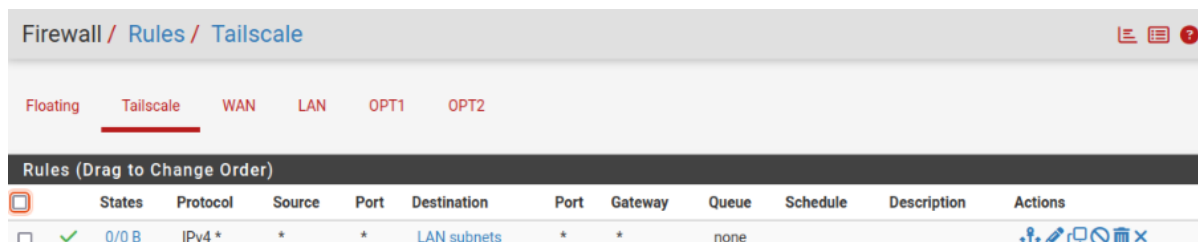
Hình 4. Cấu hình định tuyến Tailscale trên pfSense

- Vào VPN -> Tailscale -> Authentication, sao chép khóa xác thực vào mục Pre-authentication Key, có thể tạo được ở đường dẫn <https://login.tailscale.com/admin/settings/keys> của Tailscale:



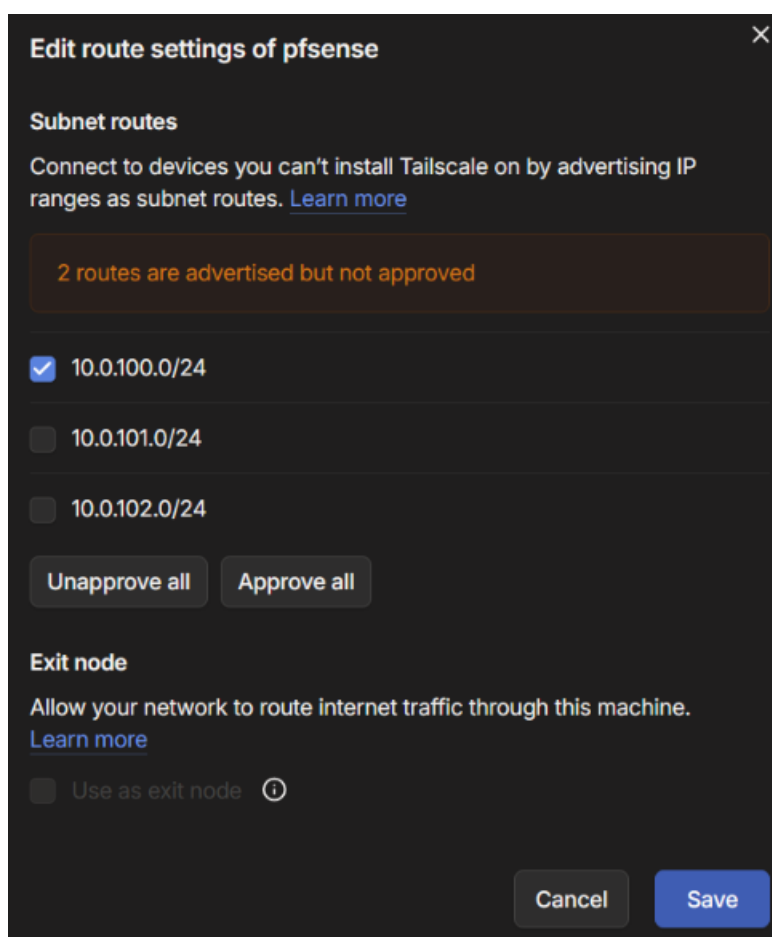
Hình 5. Cấu hình xác thực pfSense với Tailscale

- Mặc định pfSense sẽ chặn hết tất cả lưu lượng từ bên ngoài đi vào, nên nếu muốn máy cài ELK Stack và TheHive kết nối được với máy chủ OSSEC, cần cài một rule cho phép hai máy trên truy cập được (ở đây để dễ dàng cấu hình thì tạm thời thêm rule cho phép tất cả lưu lượng) ở Firewall -> Rules -> Tailscale:



Hình 6. Cấu hình rule cho kết nối thông qua Tailscale

- Quay lại website Tailscale quản lý các máy đã cài đặt Tailscale, tìm pfSense và cấu hình định tuyến, cho phép quảng bá vùng mạng của máy chủ OSSEC là 10.0.100.0/24:



Hình 7. Cấu hình quảng bá vùng mạng trên Tailscale

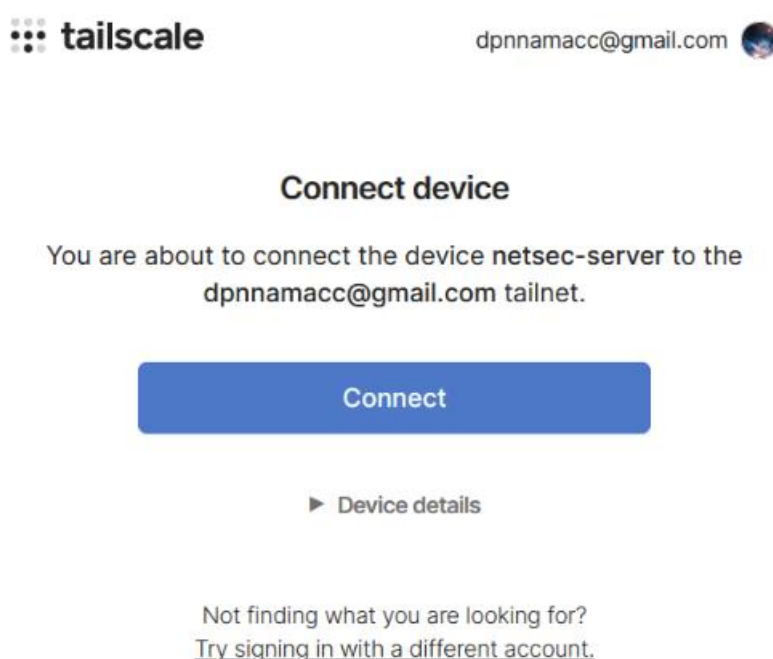
Cài đặt ở máy được cài ELK Stack:

- Máy được cài ELK Stack chạy trên hệ điều hành Ubuntu, nên có thể thực hiện những lệnh này và cài đặt Tailscale nhanh chóng (sẽ có đường dẫn xuất hiện để xác thực vào vùng mạng quản lý, sao chép vào nơi đã đăng nhập Tailscale để hoàn tất quá trình cài đặt):

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up --accept-routes
```

Bảng 8. Cài đặt Tailscale trên máy cài ELK Stack

Cài đặt ở máy được cài TheHive: Vì máy được cài TheHive chạy trên hệ điều hành Windows, sẽ có một tệp thực thi để trực tiếp Tailscale. Sau đó, tiến hành đăng nhập bằng tài khoản chính để xác nhận máy vào cùng vùng mạng quản lý.

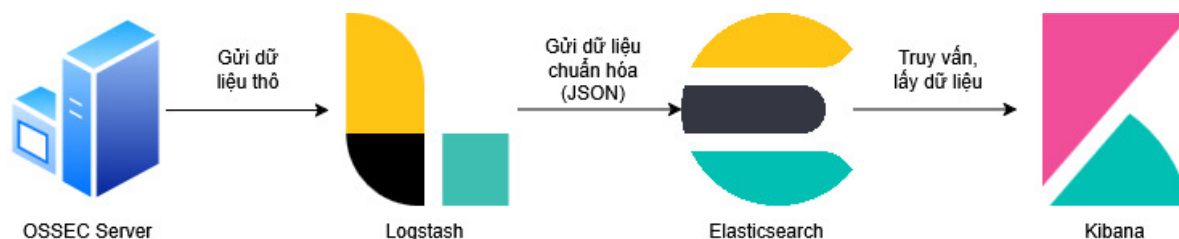


Hình 8. Màn hình xác nhận máy đăng ký vào vùng mạng quản lý của Tailscale

2.3. ELK Stack

Sơ đồ hoạt động của ELK Stack có thể được mô tả ở hình dưới, và cũng là mạch hoạt động mà nhóm sẽ dựa vào để cấu hình chi tiết cho ELK Stack, cụ thể:

- Dữ liệu ban đầu được lấy từ máy chủ OSSEC và gửi về cho Logstash.
- Logstash nhận dữ liệu gốc và chuẩn hóa lại dưới dạng JSON, phù hợp để Elasticsearch có thể xử lý.
- Elasticsearch nhận dữ liệu đã chuẩn hóa, tiến hành lưu trữ và phân tích dữ liệu.
- Kibana thực hiện truy vấn, và trực quan hóa dữ liệu được trích xuất ra.



Hình 9. Hoạt động của ELK Stack

Trước khi tiến hành cài đặt, cần phải thêm thư viện mã nguồn của Elastic, để có thể thực hiện việc cài đặt ELK Stack:

```
curl -fsSL https://artifacts.elastic.co/GPG-KEY-elasticsearch | \
sudo gpg --dearmor -o /usr/share/keyrings/elastic.gpg

echo "deb [signed-by=/usr/share/keyrings/elastic.gpg] \
https://artifacts.elastic.co/packages/8.x/apt stable main" | \
sudo tee /etc/apt/sources.list.d/elastic.list
```

Bảng 9. Lệnh thêm thư viện mã nguồn của Elastic

Sau đó, tiến hành cài đặt lần lượt Elasticsearch, Logstash và Kibana:

```
sudo apt update
sudo apt install elasticsearch
sudo apt install kibana
sudo apt install logstash
```

Bảng 10. Lệnh cài đặt elasticsearch

2.3.1. Cấu hình Elasticsearch

Tìm và chỉnh sửa các thông tin sau ở tệp cấu hình /etc/elasticsearch/elasticsearch.yml, trong đó:

- Cần thiết lập network.host bằng 0.0.0.0 để các máy khác có thể tìm thấy.
- Tắt các tính năng bảo mật để dễ dàng tinh chỉnh và sửa lỗi (xpack.security.enabled và xpack.security.enrollment.enabled).
- Vì chỉ dùng một máy ảo duy nhất, cần thêm dòng discovery.type để cho Elasticsearch biết để không tìm các máy khác, đồng thời xóa đi dòng cluster.initial_master_nodes:

```
cluster.name: <tên cluster>
node.name: <tên node>
network.host: 0.0.0.0
xpack.security.enabled: false
xpack.security.enrollment.enabled: false
discovery.type: single-node
#cluster.initial_master_nodes:
```

Bảng 11. Cấu hình Elasticsearch

Ngoài ra, cần phải giới hạn tài nguyên Elasticsearch có thể sử dụng để phân tích dữ liệu, ở đây sẽ giới hạn 1GB RAM ở tệp `/etc/elasticsearch/jvm.options.d/heap.options` để tránh làm sập hệ thống:

```
-Xms1g
-Xmx1g
```

Bảng 12. Cấu hình tài nguyên mà Elasticsearch được phép sử dụng

2.3.2. Cấu hình Logstash

Khi nhận dữ liệu từ máy chủ OSSEC, dữ liệu nhận vào sẽ không hẳn được định dạng dưới dạng JSON, vì vậy, ngoài việc mở cổng lắng nghe tới máy chủ OSSEC với cổng 5140 (mặc định), ta cũng cần cấu hình làm sạch dữ liệu đầu vào để Elasticsearch có thể tách trường dữ liệu chuẩn xác, cụ thể ở tệp `/etc/logstash/conf.d/ossec.conf` như sau:

- input: Mở cổng lắng nghe tới OSSEC bằng cổng 5140, đồng thời định nghĩa kiểu dữ liệu là “ossec” để có thể tiến hành lọc và làm sạch dữ liệu.
- filter: Làm sạch dữ liệu khi nhận dữ liệu kiểu “ossec”, loại bỏ phần đầu của dữ liệu (“ossec:”) để có dữ liệu dạng JSON chuẩn, và chuyển dữ liệu trích xuất ra được thành đối tượng JSON chuẩn cho Elasticsearch. Ngoài ra, tiến hành loại bỏ một số trường không cần thiết, vốn chứa toàn bộ nội dung OSSEC gửi ban đầu, cũng như nội dung trích xuất gốc vì đã có đối tượng JSON chuẩn sẽ được gửi sau đó.
- output: Gửi dữ liệu chuẩn hóa về Elasticsearch, kèm thông tin debug.

```
input {
  # Lắng nghe log từ OSSEC gửi sang qua cổng UDP 5140
  udp {
    port => 5140
    type => "ossec"
  }
}

filter {
  if [type] == "ossec" {

    # 1. Tách phần mào đầu Syslog ra, chỉ lấy phần ngoặc nhọn JSON trở đi
    grok {
      match => { "message" => "ossec: %{GREEDYDATA:json_payload}" }
    }

    # 2. Parse phần JSON vừa lấy được
    if [json_payload] {
      json {
        source => "json_payload"
      }
    }
  }
}
```

```

    }
  }

  # 3. Dọn dẹp rác: Xóa bỏ các trường thô để tiết kiệm ổ cứng và làm sạch Kibana
  mutate {
    remove_field => ["message", "json_payload", "event.original"]
  }
}

output {
  if [type] == "ossec" {
    elasticsearch {
      # Logstash nằm cùng máy với Elasticsearch nên để localhost
      hosts => ["http://localhost:9200"]
      index => "ossec-alerts"
    }
    stdout { codec => rubydebug }
  }
}

```

Bảng 13. Cấu hình Logstash

2.3.3. Cấu hình Kibana

Tìm và chỉnh sửa các thông tin sau ở tệp cấu hình `/etc/kibana/kibana.yml` để cho phép máy bên ngoài truy cập vào Kibana:

```
server.host: "0.0.0.0"
```

Bảng 14. Cấu hình Kibana

2.3.4. Cấu hình máy chủ OSSEC

Dữ liệu ở máy chủ OSSEC cần biết được phải chuyển dữ liệu về Logstash, vì vậy, cần thêm thẻ `<syslog_output>` trong tệp cấu hình `/var/ossec/etc/ossec.conf`, chú ý ở đây phải cần chuyển dưới định dạng JSON để Elasticsearch có thể đọc và phân tích dữ liệu nhận vào:

```

<ossec_config>
...
  <syslog_output>
    <server>100.84.47.86</server>
    <port>5140</port>
    <format>json</format>
  </syslog_output>
...
</ossec_config>

```

Bảng 15. Cấu hình máy chủ OSSEC để gửi dữ liệu về Logstash

2.3.5. Triển khai ELK Stack

Sau khi hoàn tất cấu hình, tiến hành triển khai toàn bộ ELK Stack với lệnh sau:

```
sudo systemctl enable elasticsearch
sudo systemctl start elasticsearch

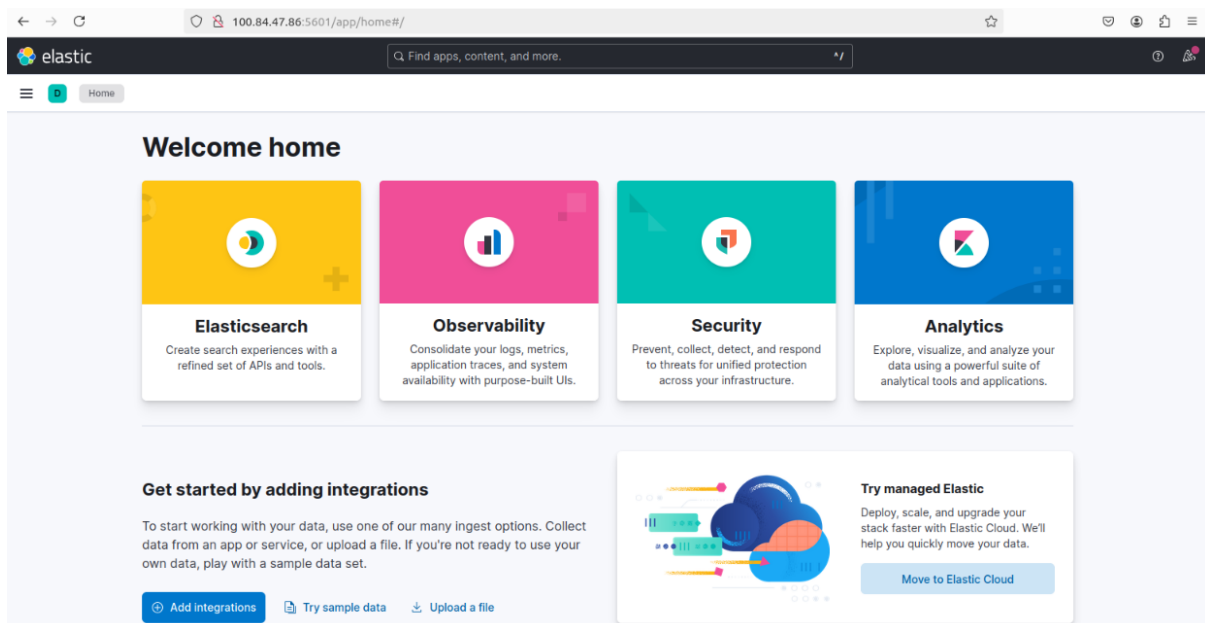
sudo systemctl enable logstash
sudo systemctl start logstash

sudo systemctl enable kibana
sudo systemctl start kibana
```

Bảng 16. Lệnh triển khai ELK Stack

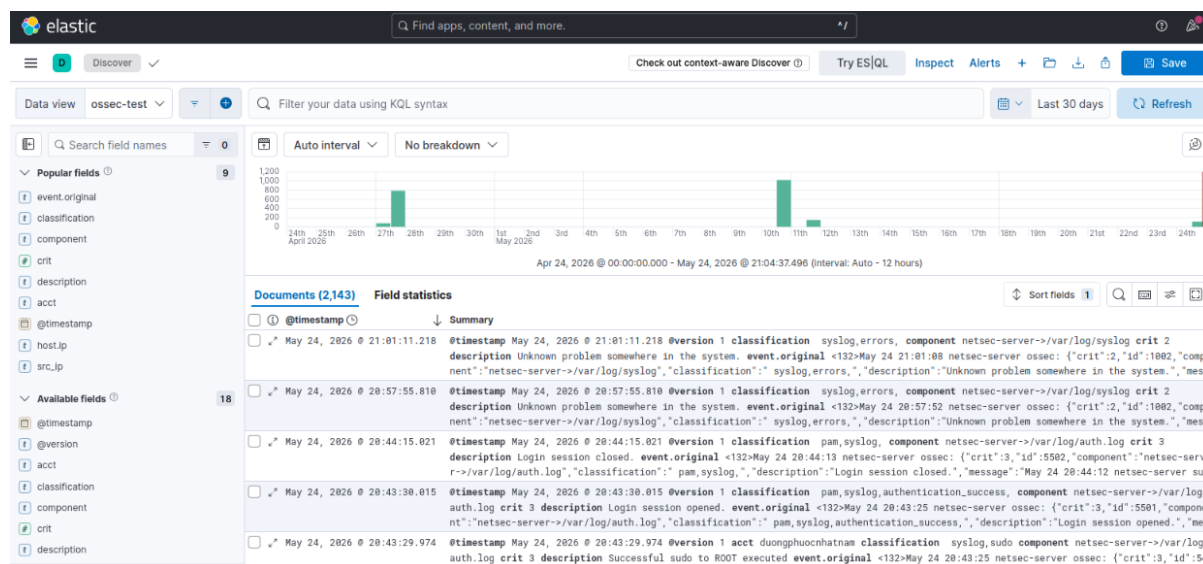
2.3.6. Cấu hình giao diện web

Sau khi triển khai thành công, truy cập vào <http://<IP-ELK>:5601> để vào giao diện của Kibana:



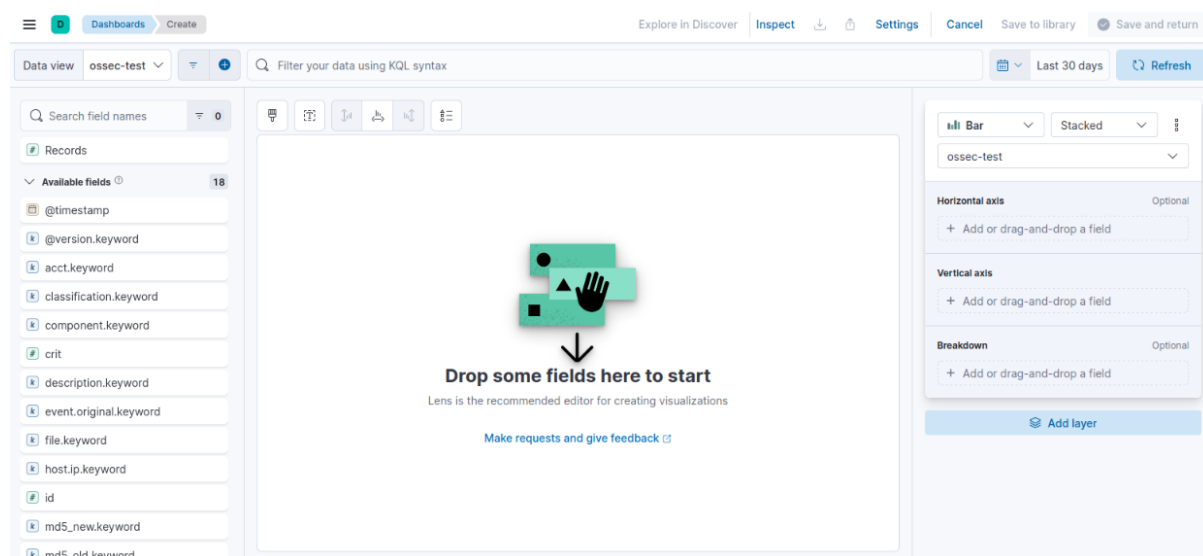
Hình 10. Giao diện của Kibana

- Truy cập vào Analytics -> Discover để thấy dữ liệu của OSSEC trên Kibana:



Hình 11. Dữ liệu OSSEC gửi về trên Kibana

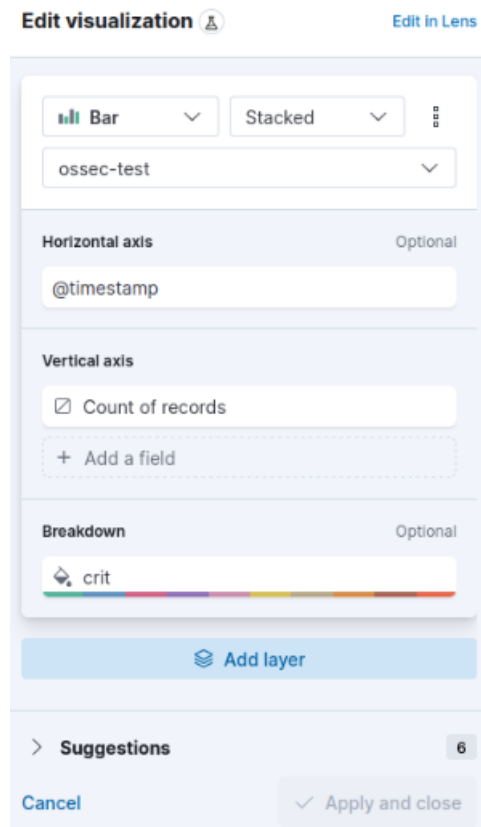
Để có thể biết được tình hình bảo mật của hệ thống, tiến hành vào Analytics -> Dashboard -> Create Visualization để có thể tạo bảng thống kê tình hình hệ thống:



Hình 12. Giao diện tạo bảng

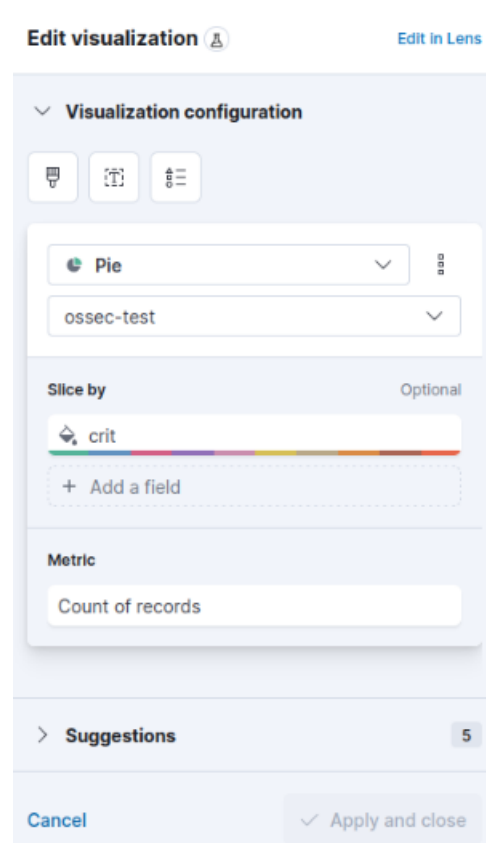
Các nội dung mà chúng ta sẽ tạo bao gồm:

- Số lượng cảnh báo diễn ra theo thời gian, phân loại theo mức độ nghiêm trọng:
 - o Chọn biểu đồ cột (Bar), thuộc loại Stacked.
 - o Trên mục Horizontal axis (Trục hoành), chọn trường @timestamp.
 - o Ở mục Vertical axis (Trục tung), chọn hàm Count of records để đếm số lượng cảnh báo trong mốc thời gian cụ thể.
 - o Ở mục Breakdown, chọn trường crit.



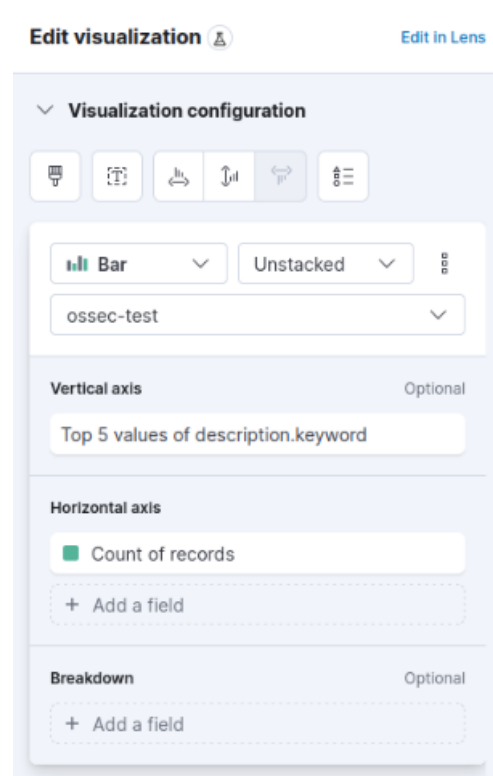
Hình 13. Cấu hình biểu đồ cột

- Phân bố mức độ nghiêm trọng của cảnh báo:
 - Chọn biểu đồ tròn (Pie).
 - Ở mục Slice by, chọn crit (trường mức độ nghiêm trọng của cảnh báo).
 - Ở mục Count by, chọn Count of records.



Hình 14. Cấu hình biểu đồ tròn

- Thẻ loại cảnh báo được phát ra nhiều nhất:
 - Chọn biểu đồ cột (Bar), thuộc loại Unstacked.
 - Tại Vertical axis, chọn trường description.keyword (thẻ loại cảnh báo được phát ra).
 - Tại Horizontal axis, chọn Count of records.



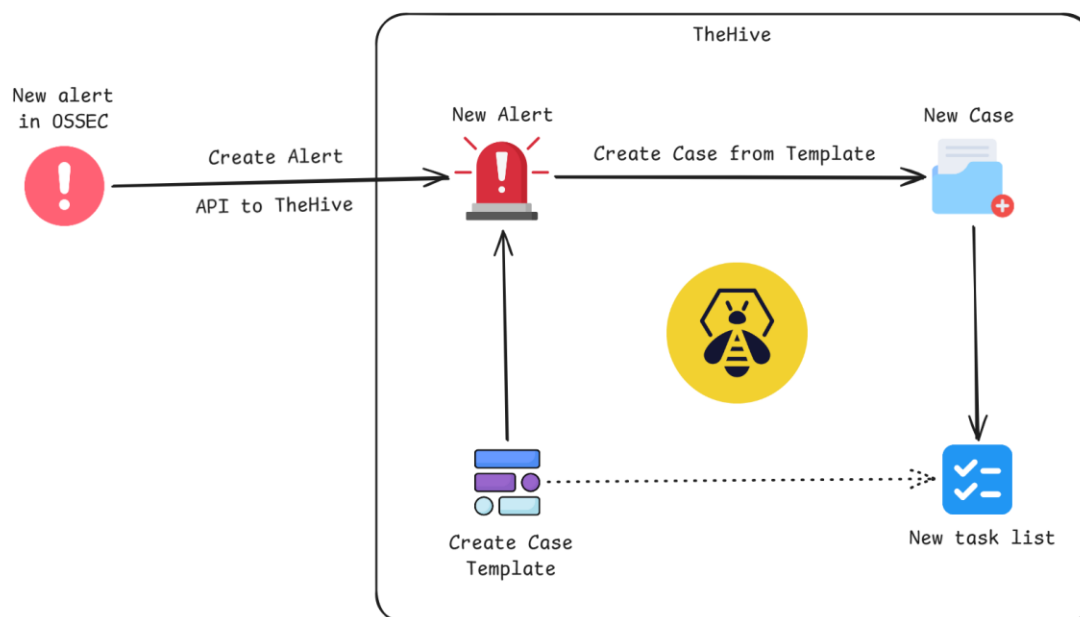
Hình 15. Cấu hình biểu đồ cột ngang

- Bảng chứa nội dung đầy đủ của cảnh báo:
 - Quay lại Analytics -> Discover.
 - Tại mục bên trái hiển thị danh sách các trường dữ liệu, chọn các trường mong muốn thêm vào bảng.
 - Nhấn Save để lưu bảng đã được chỉnh sửa.

Sau khi cấu hình thành công, lần lượt thêm các bảng vào trang chủ để có thể hiển thị tình hình bảo mật hiện tại của hệ thống một cách trực quan.

2.4. TheHive

Sau khi đã có cảnh báo tấn công từ OSSEC, yêu cầu về một công cụ để quản lý các trường hợp cảnh báo tấn công là cần thiết. TheHive chính là giải pháp hiệu quả nhất cho vấn đề trên.



Hình 16. Quy trình xử lý của TheHive

Đầu tiên, dựng một tệp docker-compose.yml cho TheHive 5.2 Community Edition. TheHive không hoạt động độc lập mà đòi hỏi một hệ sinh thái các dịch vụ để xử lý khối lượng dữ liệu lớn. Khối kiến trúc tiêu chuẩn nhóm sử dụng bao gồm:

- Cassandra (Cơ sở dữ liệu chính): Sử dụng để lưu trữ toàn bộ dữ liệu có cấu trúc của hệ thống như thông tin về Case, Alert, Observable, Task. Nhóm dùng bản cassandra:4.0.
- Elasticsearch (Công cụ tìm kiếm): Chịu trách nhiệm lập chỉ mục (index) mọi dữ liệu trong TheHive, cho phép hệ thống truy vấn và tìm kiếm với tốc độ cao. Nhóm đang chạy bản elasticsearch:7.17.9.

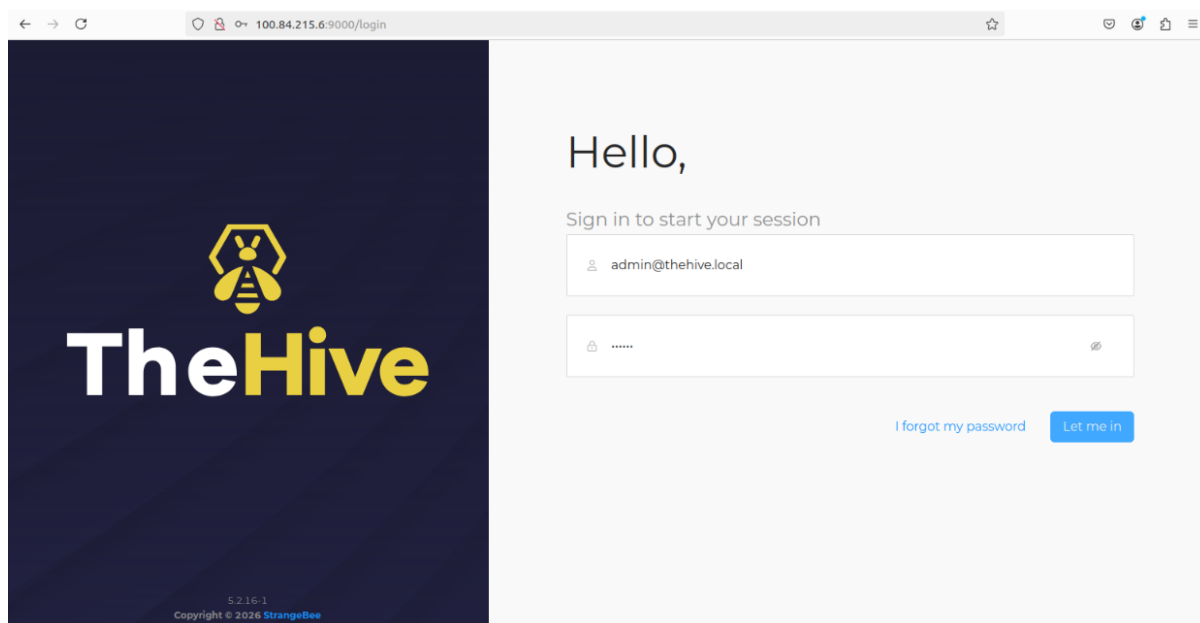
Name	Container ID	Image	Port(s)	CPU (%)	Memory usage	Memory (%)	Disk read/write	Network I/O	PIDS	Last started	Actions
desktop	-	-	-	313.64%	2.46GB / 11.47GB	64.28%	443MB / 37.47M	6.18KB / 37KB	46, 93, 29	13 seconds ago	[Stop] [Refresh] [Restart]
thehive	c224f0b0b64	thehive/thehive:5.2	9000/9000	187.46%	373.0MB / 3.82GB	9.54%	137MB / 1.57MB	2.08KB / 126B	46	13 seconds ago	[Stop] [Refresh] [Restart]
cassandra	213cd879433	cassandra:4.0	-	9.72%	1.44GB / 3.82GB	37.35%	193MB / 35.3MB	26B / 126B	93	13 seconds ago	[Stop] [Refresh] [Restart]
elasticsearch	353bd759304	elasticsearch:7.17.9	-	146.48%	672.7MB / 3.82GB	17.19%	113MB / 610KB	2.08KB / 126B	29	13 seconds ago	[Stop] [Refresh] [Restart]

Hình 17. Khối kiến trúc Docker chạy TheHive

Khối kiến trúc trên khởi chạy TheHive nội bộ trên máy chủ ở cổng 9000. Khi truy cập qua trình duyệt, thu được giao diện đăng nhập TheHive. TheHive cung cấp một tài khoản admin mặc định là:

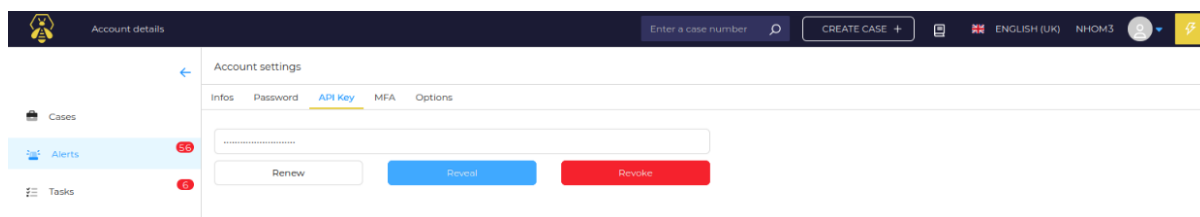
- Username: admin@thehive.local

- Password: secret.



Hình 18. Giao diện đăng nhập TheHive

Như đã nói ở phần mô tả, TheHive là một công cụ được xây dựng với tiêu chí API là trên hết. Mọi thao tác trên giao diện web đều có thể được gọi thông qua API. Vậy nên, để đưa cảnh báo từ OSSEC vào TheHive, nhóm đã xây dựng một đoạn mã Python thực hiện chức năng trên. TheHive có một hệ thống các vai trò, tiêu biểu là admin (quản trị viên) với quyền chính là quản lý người dùng; và analyst (phân tích viên) với quyền chủ yếu là thao tác trên Cảnh báo (Alert) và Trường hợp (Case). API key đưa vào đoạn mã của nhóm là của tài khoản analyst, và có thể được tìm thấy ở phần Account Setting.



Hình 19. API Key của tài khoản TheHive

```
#!/usr/bin/env python3
import re
import time
import requests

LOG_FILE = "/var/ossec/logs/alerts/alerts.log"
THEHIVE_URL = "http://100.84.215.6:9000/api/v1/alert"
```

```

API_KEY = "<THEHIVE-API-KEY>"

HEADERS = {
    "Authorization": f"Bearer {API_KEY}",
    "Content-Type": "application/json"
}

seen_ids = set()

# =====
# PARSER
# =====
def parse_block(block):
    lines = [l.strip() for l in block.split("\n") if l.strip()]

    if len(lines) < 3:
        print("[DEBUG] Block too short")
        return None

    print("\n[DEBUG] RAW BLOCK:\n", block)

    # dòng 1
    m1 = re.match(r"\*\* Alert (\d+\.\d+): - (.*)", lines[0])
    if not m1:
        print("[DEBUG] FAIL line1:", lines[0])
        return None

    alert_id = m1.group(1)
    groups = m1.group(2)

    # dòng 2
    m2 = re.match(r"(.+?) (\S+)->(.+)", lines[1])
    if not m2:
        print("[DEBUG] FAIL line2:", lines[1])
        return None

    timestamp = m2.group(1)
    hostname = m2.group(2)

    # dòng 3
    m3 = re.match(r"Rule: (\d+) \(level (\d+)\) -> '(.*)'", lines[2])
    if not m3:
        print("[DEBUG] FAIL line3:", lines[2])
        return None

    rule_id = m3.group(1)

```

```

level = int(m3.group(2))
description = m3.group(3)

full_log = "\n".join(lines)

return {
    "id": alert_id,
    "level": level,
    "description": description,
    "full_log": full_log,
    "hostname": hostname,
    "timestamp": timestamp,
    "rule_id": rule_id
}

# =====
# SEND TO THEHIVE
# =====
def send_to_thehive(alert):
    print(f"[DEBUG] Processing alert {alert['id']} level={alert['level']}")

    if alert["id"] in seen_ids:
        print("[DEBUG] Duplicate alert skipped")
        return

    seen_ids.add(alert["id"])

    severity = max(1, min((alert["level"] // 4) + 1, 4))

    data = {
        "title": alert["description"],
        "description": alert["full_log"],
        "severity": severity,
        "source": "ossec",
        "sourceRef": alert["id"],
        "type": "external"
    }

    print("[DEBUG] Sending payload:", data)

    try:
        r = requests.post(THEHIVE_URL, headers=HEADERS, json=data, timeout=5)

        print("[DEBUG] STATUS:", r.status_code)
        print("[DEBUG] RESPONSE:", r.text)

```

```

        if r.status_code == 201:
            print(f"[+] Sent alert {alert['id']}")
        else:
            print(f"[!] Failed to send alert {alert['id']}")

    except Exception as e:
        print("[ERROR] Request failed:", e)

# =====
# FOLLOW FILE (tail -f)
# =====
def follow(file):
    file.seek(0, 2) # jump to EOF
    buffer = ""

    while True:
        line = file.readline()

        if not line:
            time.sleep(0.2)
            continue

        line = line.rstrip()

        if line.startswith("** Alert"):
            if buffer:
                yield buffer
                buffer = line + "\n"
            else:
                buffer += line + "\n"

# =====
# MAIN
# =====
def main():
    print("[*] Starting OSSEC → TheHive forwarder...")

    with open(LOG_FILE, "r") as f:
        for block in follow(f):
            alert = parse_block(block)

            if alert:
                send_to_thehive(alert)

```



```

else:
    print("[DEBUG] Alert parsing failed")

if __name__ == "__main__":
    main()

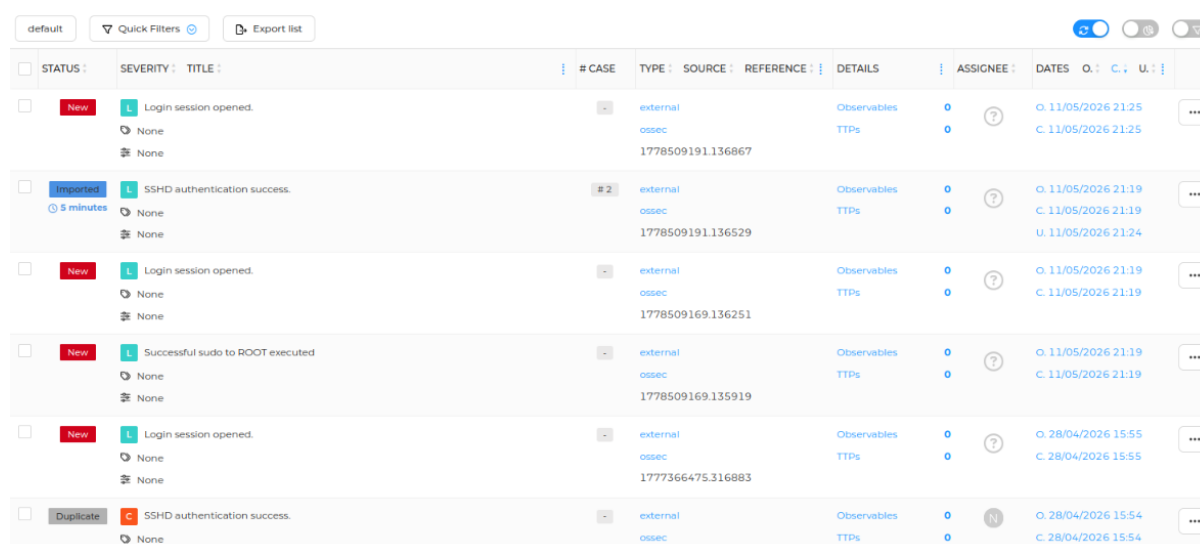
```

Bảng 17. Script gửi cảnh báo từ máy chủ OSSEC sang TheHive

Đoạn mã trên đóng vai trò là một trình chuyên tiếp log. Chức năng chính của nó là tự động giám sát, trích xuất các cảnh báo bảo mật từ hệ thống OSSEC và đẩy trực tiếp lên nền tảng TheHive. Cơ bản hoạt động của nó gồm 3 giai đoạn:

- **Giám sát:** Kịch bản này mở tệp nhật ký của OSSEC (/var/ossec/logs/alerts/alerts.log) và di chuyển con trỏ xuống cuối tệp. Nó liên tục đọc các dòng mới được thêm vào (tương tự tail -f), khi nhận diện được chuỗi “Alert” (Chuỗi bắt đầu của một cảnh báo), nó xác định đây là một cảnh báo mới, tiến hành gom tất cả các dòng thuộc cảnh báo đó thành một khối dữ liệu (block) duy nhất để xử lý.
- **Phân tích:** Nhận khối dữ liệu thô và đọc hiểu 3 dòng đầu tiên của log. Trích xuất các thông tin cốt lõi bao gồm: alert_id (Mã cảnh báo), level (Mức độ), description (Mô tả), timestamp (Thời gian), hostname (Máy chủ bị ảnh hưởng) và rule_id (Mã luật vi phạm).
- **Gửi lên TheHive:** Kịch bản kiểm tra xem ID của cảnh báo đã từng được gửi đi chưa thông qua danh sách seen_ids để lọc trùng lặp. Sau đó, chuyển đổi thang đo mức độ cảnh báo của OSSEC sang thang đo mức độ nghiêm trọng (severity từ 1 đến 4) tương thích với TheHive. Cuối cùng, đóng gói toàn bộ thông tin thành một tệp JSON và gửi POST request qua REST API của TheHive để tự động tạo một Alert mới trên hệ thống.

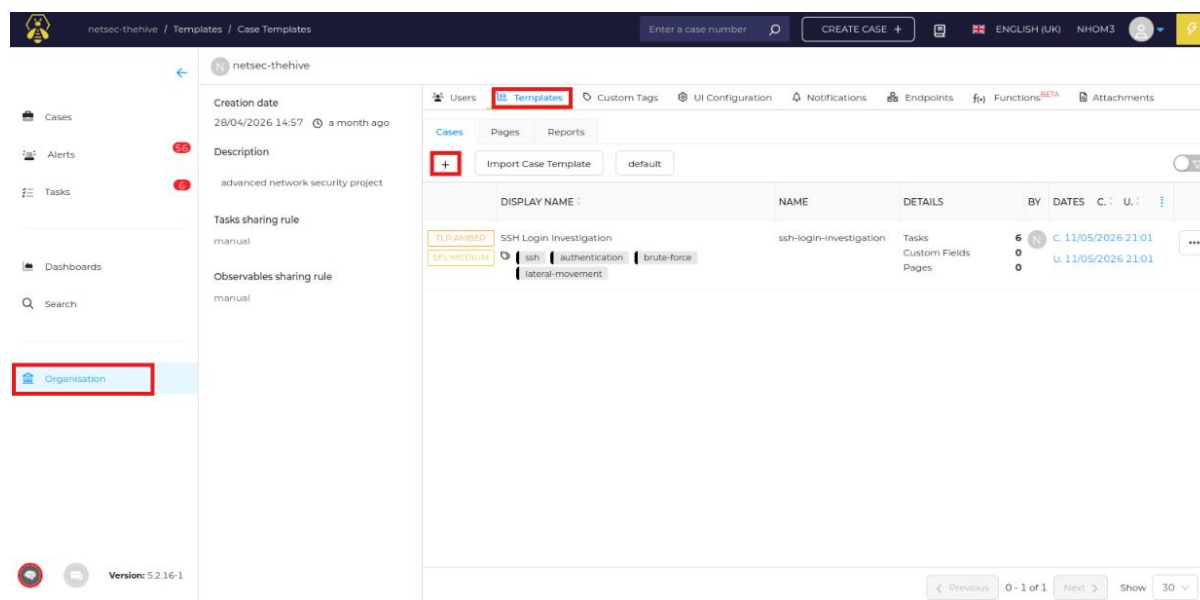
Sau khi gửi cảnh báo qua API, bên phía giao diện web của TheHive, những cảnh báo mới nhất sẽ được tạo và hiển thị trong phần Alert.



STATUS	SEVERITY	TITLE	# CASE	TYPE	SOURCE	REFERENCE	DETAILS	ASSIGNEE	DATES	O.	C.	U.
None	Low	Login session opened.	-	external	ossec	1778509191.136867	Observables TTPs	0 0	?	O: 11/05/2026 21:25 C: 11/05/2026 21:25		
Imported	Low	SSH authentication success.	#2	external	ossec	1778509191.136529	Observables TTPs	0 0	?	O: 11/05/2026 21:19 C: 11/05/2026 21:19 U: 11/05/2026 21:24		
None	Low	Login session opened.	-	external	ossec	1778509169.136251	Observables TTPs	0 0	?	O: 11/05/2026 21:19 C: 11/05/2026 21:19		
None	Low	Successful sudo to ROOT executed	-	external	ossec	1778509169.135919	Observables TTPs	0 0	?	O: 11/05/2026 21:19 C: 11/05/2026 21:19		
None	Low	Login session opened.	-	external	ossec	1777366475.316883	Observables TTPs	0 0	?	O: 28/04/2026 15:55 C: 28/04/2026 15:55		
Duplicate	Low	SSH authentication success.	-	external	ossec		Observables TTPs	0 0	N	O: 28/04/2026 15:54 C: 28/04/2026 15:54		

Hình 20. Các cảnh báo được truyền tới TheHive

Với yêu cầu đề xuất những bước phản hồi đối với các cảnh báo, nhóm sử dụng tính năng Case Template của TheHive. Case Template là một tính năng phụ của TheHive cho phép tạo các trường hợp (Case) mẫu ứng với mỗi loại tấn công hay sự cố bảo mật cụ thể. Trong mẫu này, người dùng có thể tự liệt kê các hành động (Task) để ứng phó với kịch bản đó. Để tạo được Case Template, cần sử dụng tài khoản là analyst, vào Organisation → Templates, nhấn vào biểu tượng (+) để tạo Case Template mới.



Hình 21. Tạo Case Template

Tại giao diện tạo Case Template, ngoài các trường cơ bản như tên hay mô tả, cần chú ý tới phần Tasks. Tại đây, nếu muốn thêm một bước trong quy trình phản hồi lại trường hợp tấn công, nhấn vào biểu tượng (+), điền tên và mô tả của hành động. Sau khi tạo, một Task mới sẽ được thêm vào phía dưới của danh sách các Task ứng với Case Template đó.

Nhóm thực hiện xây dựng một Case Template SSH Login Investigation, với mục đích làm mẫu cho các trường hợp tấn công thông qua SSH, với các Task được thể hiện trong ảnh.

Editing a Case Template

PAP

PAP: CLEAR PAP: GREEN **PAP: AMBER** PAP: RED

Severity

LOW **MEDIUM** HIGH CRITICAL

Tags

ssh authentication brute-force lateral-movement

* Description

investigating abnormal ssh logins

Tasks

Investigation - Collect SSH Log Evidence	Edit	Delete
Investigation - Analyze Source IP Address	Edit	Delete
Investigation - Review Compromised Account Activity	Edit	Delete
Containment - Determine Attack Scope	Edit	Delete
Remediation - Containment and Remediation	Edit	Delete
Reporting - Write Incident Report	Edit	Delete

Cancel Confirm case template edition

Hình 22. Giao diện tạo và chỉnh sửa Case Template

Sau khi đã tạo được Case Template, quay trở lại mục Alerts, nhấn vào Alert muốn xử lý và chọn vào phần CREATE CASE ở góc trên bên phải để tạo một Case dựa trên Template vừa tạo.



Hình 23. Tạo Case từ Alert

Ở giao diện tạo Case, nhấn chọn vào button dropdown và chọn Case Template vừa tạo.

Create case

How do you want to create your case?

Empty case

From template

From archive (.tar)

From MISP (.json)

Hình 24. Chọn Case Template cho Case muốn tạo

Tại giao diện tạo Case từ Case Template, nhập vào các trường yêu cầu bao gồm: Title (Tiêu đề Case), Date (Ngày tạo, TheHive tự động điền), Severity (Mức độ nghiêm trọng), TLP (Độ nhạy cảm của thông tin, để mặc định là Amber), PAP (Hành động cho phép với thông tin, để mặc định là Amber), Tags (Nhãn, TheHive tự động sao chép từ CaseTemplate), Description (Mô tả của Case, TheHive tự sao chép phần mô tả của Alert vào đây). Cuối cùng, nhấn nút tạo Case.

CHƯƠNG V. KẾT QUẢ THỰC NGHIỆM

1. OSSEC

1.1. Active Response

Thực hiện tấn công bruteforce SSH với script bash brute_ssh.sh:

```
#!/bin/bash

TARGET="10.0.102.102"
USER="labuser"

PASSWORDS=(
    "admin123"
    "password"
    "111111"
    "qwerty"
    "labuser123"
    "root"
    "123123"
    "12345678"
    "letmein"
    "123456"
)

echo "Bắt đầu tấn công Brute-force vào $USER@$TARGET..."
echo "-----"

for pass in "${PASSWORDS[@]}; do
    echo "[*] Đang thử mật khẩu: $pass"

    sshpass -p "$pass" ssh -o StrictHostKeyChecking=no -o ConnectTimeout=3
    $USER@$TARGET "exit" 2>/dev/null

    if [ $? -eq 0 ]; then
        echo "[+] THÀNH CÔNG! Mật khẩu là: $pass"
        break
    else
        echo "[-] Thất bại."
    fi

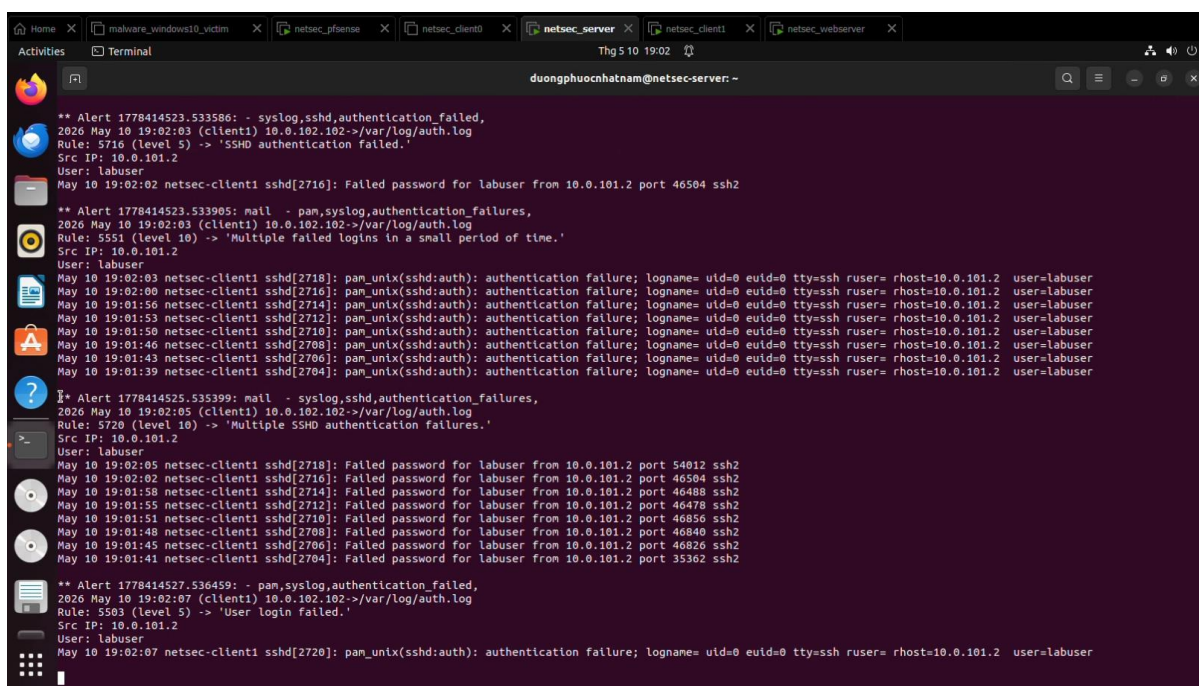
    sleep 1
done
```

Bảng 18. Chương trình kiểm thử kịch bản Active Response



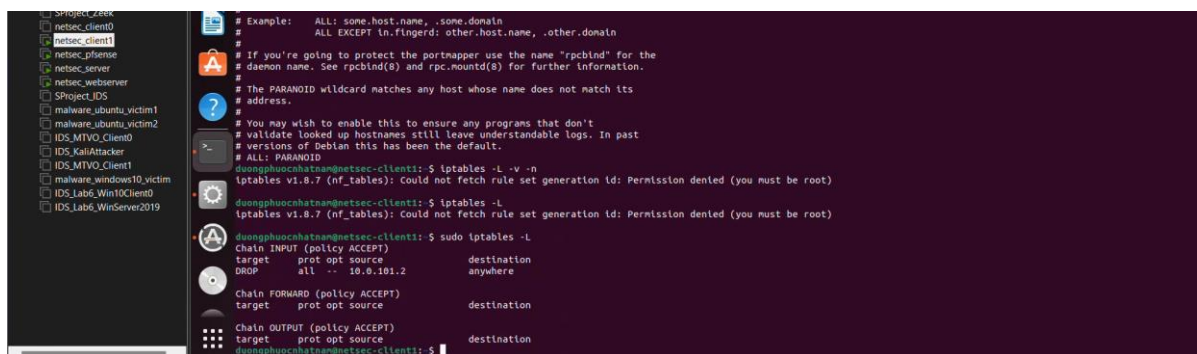
Hình 26. Chương trình kiểm thử được thực hiện và bị chặn bởi OSSEC

Ta quan sát thấy log liên tục ghi nhận sự kiện tài khoản labuser đăng nhập thất bại. Ngay khi số lần thử sai vượt ngưỡng, hệ thống đã phát hiện hành vi Brute-force và đẩy mức độ cảnh báo lên cao nhất: kích hoạt **Rule 5720 - Level 10**.



Hình 27. Nhật ký sự kiện của OSSEC chủ động chặn tấn công bruteforce

Trong kiến trúc của OSSEC, Rule 5720 chính là **điều kiện kích hoạt** cho tính năng Active Response đã được nhóm cấu hình ở bước trước. Khi Rule 5720 nổ ra, OSSEC không chỉ dừng lại ở việc ghi log thụ động, mà hệ thống đã ngay lập tức tự động gọi kịch bản thực thi firewall-drop.sh (chặn IP bằng tường lửa iptables) ngầm ở bên dưới.

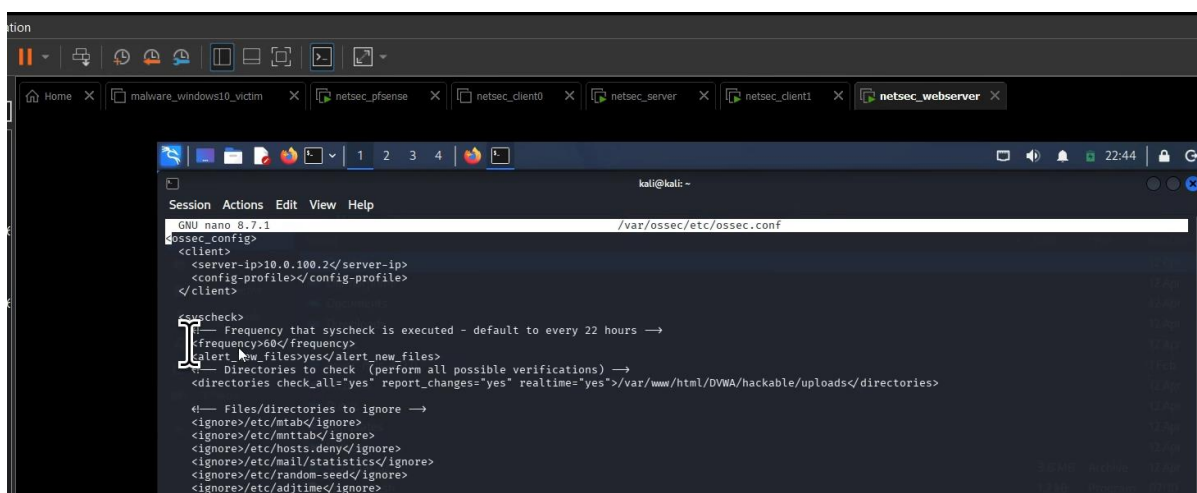


Hình 28. Thông tin chặn IP ở iptables

Hệ quả trực tiếp của hành động Active Response này là địa chỉ IP của máy tấn công (10.0.101.2) lập tức bị đưa vào danh sách cấm (Drop). Điều này giải thích tại sao chương trình khai thác tự động của kẻ tấn công bị chặn đứng, ngắt kết nối hoàn toàn và không thể tiếp tục thử các mật khẩu tiếp theo trong danh sách. Qua đó, minh chứng rõ ràng khả năng tự động phòng ngự và ngăn chặn rủi ro tức thời của tính năng Active Response.

1.2. File Integrity checking

Sau khi cấu hình file OSSEC như ở phần mô tả cấu hình với mục đích giảm thời gian syscheck nhằm tạo điều kiện thuận lợi với mục đích kiểm tra kết quả, giám sát cả việc thêm tệp, thêm đường dẫn thư mục /var/www/html/DVWA/hackable/uploads vào thư mục cần giám sát:



Hình 29. Cấu hình trên máy chủ OSSEC

Ta thêm tệp vào đường dẫn tải lên của DWVA, thực hiện khởi động lại OSSEC và chờ tới khi nó thực hiện quét syscheck xong, sau đó ta thêm nội dung vào tệp chotha.txt hiện tại nhằm mục đích sửa đổi tệp.


```
(kali@kali)-[~]
$ echo 'Nội dung gọc' | sudo tee /var/www/html/DVWA/hackable/uploads/chotha.txt
Nội dung gọc
(kali@kali)-[~]
$ sudo systemctl restart ossec
^[[A
(kali@kali)-[~]
$ sudo tail -f /var/ossec/logs/ossec.log | grep syscheck
2026/05/10 20:28:56 ossec-syscheckd: INFO: Directory set for real time monitoring: '/var/www/html/DVWA/hackable/uploads'.
2026/05/10 20:29:58 ossec-syscheckd: INFO: Starting syscheck scan (forwarding database).
2026/05/10 20:29:58 ossec-syscheckd: INFO: Starting syscheck database (pre-scan).
2026/05/10 20:29:58 ossec-syscheckd: INFO: Initializing real time file monitoring (not started).
2026/05/10 20:29:58 ossec-syscheckd: INFO: Real time file monitoring started.
2026/05/10 20:29:58 ossec-syscheckd: INFO: Finished creating syscheck database (pre-scan completed).
2026/05/10 20:30:10 ossec-syscheckd: INFO: Ending syscheck scan (forwarding database).
^C
(kali@kali)-[~]
$ echo 'Alert 550 no ngày đi!' | sudo tee -a /var/www/html/DVWA/hackable/uploads/chotha.txt
Alert 550 no ngày đi!
```

Hình 30. Đăng tải tệp mới lên máy chủ web

Xem log của OSSEC liên quan tới file chotha.txt ta thấy có 2 rule được kích hoạt: Rule 554 về việc có tệp mới được thêm vào hệ thống và rule 550 về tính toàn vẹn của tệp chotha.txt.

```
^Cduongphuocnhatnam@netsec-server:~$ sudo tail -f /var/ossec/logs/alerts/alerts.log | grep -B 3 -A 4 "chotha.txt"
** Alert 1778421498.624061: - ossec,syscheck,
2026 May 10 20:58:18 (webserver0) 10.0.101.2->syscheck->(webserver0) 10.0.101.2->syscheck
Rule: 554 (level 5) -> 'File added to the system.'
New file '/var/www/html/DVWA/hackable/uploads/chotha.txt' added to the system.
550- Lan sua nay chac chan no 550' added to the file system.
New sha1sum is : 'f6a829b389335b0dc2aed2973923058d8936091d'
New md5sum is : 'a2a74d2f6a5748e089313cdebe40a89b'
New sha256sum is : '0cce3ece51fd3cd966ce35480f2a46f07973685ecb24323de47461402ecd5c20'

** Alert 1778421498.624679: mail - ossec,syscheck,
2026 May 10 20:58:18 (webserver0) 10.0.101.2->syscheck->(webserver0) 10.0.101.2->syscheck
Rule: 550 (level 7) -> 'Integrity checksum changed.'
Integrity checksum changed for: '/var/www/html/DVWA/hackable/uploads/chotha.txt'
Size changed from '13' to '151'
Old md5sum was: 'e117df8c99a823985b81d2705ed09f17'
New md5sum is : 'a2a74d2f6a5748e089313cdebe40a89b'
Old sha1sum was: '776411223d96228b1b8142b5eb2dfc4ed40fb5ee'
```

Hình 31. Nhật ký sự kiện của OSSEC cho kịch bản File Integrity checking

1.3. Rootkit detection

Sau khi hoàn tất config file /var/ossec/etc/ossec.conf ta thực hiện chạy lệnh bên dưới, như đã mô tả ở tiểu mục cấu hình chi tiết:

```
duongphuocnhatnam@netsec-server:~$ sudo mkdir -p /usr/lib/libsh
duongphuocnhatnam@netsec-server:~$ sudo touch /usr/lib/libsh/hide
duongphuocnhatnam@netsec-server:~$ sudo touch /dev/ptyxx
duongphuocnhatnam@netsec-server:~$
```

Hình 32. Chạy đoạn mã độc hại để kiểm thử tính năng phát hiện Rootkit

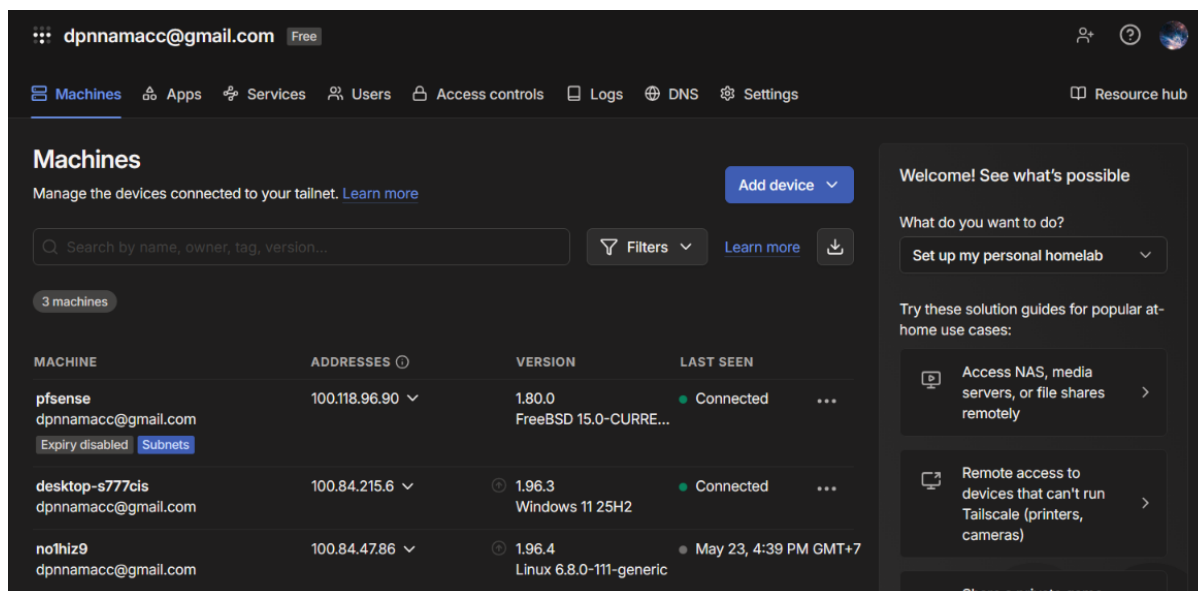
Sau đó bật quét của OSSEC và đợi một khoảng thời gian thì ta thấy được OSSEC đã phát hiện ra những tệp mà ta mới tạo trước đó.

```
duongphuocnhatnam@netsec-server:~$ sudo tail -f /var/ossec/logs/alerts/alerts.log | grep rootcheck
** Alert 1778405704.434003: mail - ossec,rootcheck,
2026 May 10 16:35:04 netsec-server->rootcheck
Rule: 510 (level 7) -> 'Host-based anomaly detection event (rootcheck).'
** Alert 1778405704.434237: mail - ossec,rootcheck,
2026 May 10 16:35:04 netsec-server->rootcheck
Rule: 510 (level 7) -> 'Host-based anomaly detection event (rootcheck).'
** Alert 1778405712.434476: mail - ossec,rootcheck,
2026 May 10 16:35:12 netsec-server->rootcheck
Rule: 510 (level 7) -> 'Host-based anomaly detection event (rootcheck).'
```

Hình 33. Nhật ký sự kiện phát hiện rootkit

2. Tailscale

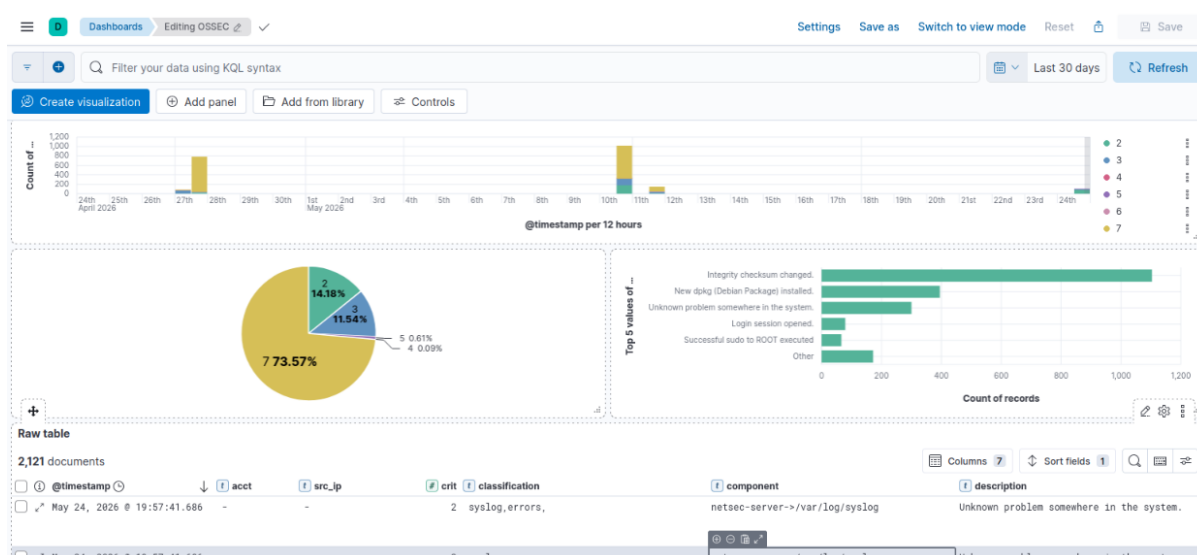
Sau khi cấu hình hoàn tất, tiến hành quay lại giao diện web của Tailscale để thấy được trạng thái của các thiết bị đã được cài đặt Tailscale. Từ đó, các thiết bị có thể kết nối được với nhau một cách bảo mật, cho phép đồ án có thể được mở rộng với bước cài đặt ELK Stack và TheHive.



Hình 34. Giao diện web hiển thị thông tin thiết bị đã cài đặt Tailscale

3. ELK Stack

Sau khi triển khai ELK Stack hoàn tất và cấu hình bảng hiển thị thông tin trực quan, ta sẽ có trang chủ có thể nói lên được tình hình bảo mật của hệ thống một cách trực quan, bao gồm bảng thống kê tình hình cảnh báo được phát ra phân loại theo mức độ nghiêm trọng theo thời gian, phân bố mức độ nghiêm trọng của cảnh báo, thống kê thể loại cảnh báo được phát ra nhiều nhất, và bảng thống kê chi tiết cảnh báo:



Hình 35. Giao diện trang chủ của Kibana sau khi cấu hình hoàn tất

4. TheHive

Chọn vào mục Case, một Case đã được tạo mới. Ở đây, có thể thấy trong tiêu đề của Case mới tạo có tiền tố SSH, đây là tiền tố nhóm đã cấu hình cho Case Template, khi tạo một Case dựa vào Case Template này, tiền tố SSH sẽ tự động thêm vào đầu Case. Ngoài ra, phần Details cho thấy có 6 Tasks ứng với 6 Tasks nhóm đã tạo với Case Template, và 1 Linked Alerts chính là Alert đã dùng để tạo Case này.

STATUS	SEVERITY	#NUMBER	TITLE	DETAILS	ASSIGNEE	DATES	S	C	U
New	#2 - SSH SSHD authentication success.			Tasks: 6	N	S. 11/05/2026 21:24			
13 days	ssh authentication brute-force lateral-movement			Observables: 0		C. 11/05/2026 21:24			
				TTPs: 0					
				Linked Alerts: 1					

Hình 36. Case mới đã được tạo

Cuối cùng, nhấn vào phần Tasks, có thể thấy tất cả các nhiệm vụ để ứng phó với trường hợp giống như đã cấu hình ở Case Template. Từ đây, có thể thực hiện phân công và xử lý mỗi nhiệm vụ để giải quyết được trường hợp. Vì TheHive không thể tự động thực hiện được chuỗi hành vi phản ứng này, nên việc xử lý có thể được xử lý thủ công, hoặc thông qua một hệ thống SOAR như Shuffle (giải pháp thường được đi cùng TheHive) hoặc n8n.

default		Quick Filters	Export list			
	GROUP : TASK :	CASE	ASSIGNEE :	DATES	S. : C. : U. :	
📄 ⚠️ ★	Reporting Write Incident Report	🔒 #2 SSH SSHD authentication success.	N	C. 11/05/2026 21:24		...
📄 ⚠️ ★	Investigation Collect SSH Log Evidence	🔒 #2 SSH SSHD authentication success.	N	C. 11/05/2026 21:24		...
📄 ⚠️ ★	Containment Determine Attack Scope	🔒 #2 SSH SSHD authentication success.	N	C. 11/05/2026 21:24		...
📄 ⚠️ ★	Investigation Analyze Source IP Address	🔒 #2 SSH SSHD authentication success.	N	C. 11/05/2026 21:24		...
📄 ⚠️ ★	Remediation Containment and Remediation	🔒 #2 SSH SSHD authentication success.	N	C. 11/05/2026 21:24		...
📄 ⚠️ ★	Investigation Review Compromised Account Activity	🔒 #2 SSH SSHD authentication success.	N	C. 11/05/2026 21:24		...

Hình 37. Danh sách các Task để xử lý trường hợp

CHƯƠNG VI. KẾT LUẬN

1. Kết quả đạt được

Nhóm đã thành công thực hiện được các yêu cầu cơ bản, cũng như mở rộng thêm phạm vi của đề tài, cụ thể:

- Nghiên cứu và hiểu rõ chức năng và nguyên tắc hoạt động của OSSEC.
- Tìm hiểu Tailscale, ELK Stack, TheHive phục vụ cho việc mở rộng hệ thống.
- Đề xuất và triển khai hệ thống SIEM và Case Management với ELK Stack và TheHive.
- Xây dựng kịch bản kiểm thử tính năng của OSSEC.

2. Hướng phát triển

Trong quá trình thực hiện đề tài, vì thời gian hạn chế nên không thể tránh được việc đồ án còn có nhiều khả năng có thể phát triển hơn, cụ thể:

- Phát triển nền tảng Case Management thành SOAR, tự động hóa việc xử lý sự cố.
- Tìm hiểu sâu hơn về việc phân tích tình hình hệ thống hiện tại với ELK Stack, từ đó có thể cải thiện hệ thống.
- Tìm hiểu cách tạo rule trên OSSEC để có thể linh hoạt đáp ứng yêu cầu bảo mật với các thiết bị đầu cuối.

NGUỒN THAM KHẢO

1. [Getting started with OSSEC.](#)
2. [ossec.conf: Syscheck Options.](#)
3. [Syscheck: FAQ.](#)
4. [What is Tailscale?](#)